TrueNorth: Design and Tool Flow of a 65 mW 1 Million Neuron Programmable Neurosynaptic Chip

Filipp Akopyan, *Member, IEEE*, Jun Sawada, *Member, IEEE*, Andrew Cassidy, *Member, IEEE*, Rodrigo Alvarez-Icaza, John Arthur, *Member, IEEE*, Paul Merolla, Nabil Imam, Yutaka Nakamura, Pallab Datta, *Member, IEEE*, Gi-Joon Nam, *Senior Member, IEEE*, Brian Taba, Michael Beakes, *Member, IEEE*, Bernard Brezzo, Jente B. Kuang, *Senior Member, IEEE*, Rajit Manohar, *Senior Member, IEEE*, William P. Risk, *Member, IEEE*, Bryan Jackson, and Dharmendra S. Modha, *Fellow, IEEE*

***Abstract*—The new era of cognitive computing brings forth the grand challenge of developing systems capable of processing massive amounts of noisy multisensory data. This type of intelligent computing poses a set of constraints, including real-time operation, low-power consumption and scalability, which require a radical departure from conventional system design. Brain-inspired architectures offer tremendous promise in this area. To this end, we developed TrueNorth, a 65 mW real-time neurosynaptic processor that implements a non-von Neumann, low-power, highly-parallel, scalable, and defect-tolerant architecture. With 4096 neurosynaptic cores, the TrueNorth chip contains 1 million digital neurons and 256 million synapses tightly interconnected by an event-driven routing infrastructure. The fully digital 5.4 billion transistor implementation leverages existing CMOS scaling trends, while ensuring one-to-one correspondence between hardware and software. With such aggressive design metrics and the TrueNorth architecture breaking path with prevailing architectures, it is clear that conventional computer-aided design (CAD) tools could not be used for the design. As a result, we developed a novel design methodology that includes mixed asynchronous–synchronous circuits and a complete tool flow for building an event-driven, low-power neurosynaptic chip. The TrueNorth chip is fully configurable in terms of connectivity and neural parameters to allow custom configurations for a wide range of cognitive and sensory perception applications. To reduce the system's communication energy, we have adapted existing application-agnostic very large-scale integration CAD placement tools for mapping logical neural networks to the physical neurosynaptic core locations on the TrueNorth chips. With that, we have successfully demonstrated the use of TrueNorth-based systems in multiple applications, including visual object recognition, with higher performance and orders of magnitude lower power consumption than the same algorithms run on von Neumann architectures. The TrueNorth chip and its tool flow serve as building blocks for future cognitive systems, and give designers an opportunity to develop novel brain-inspired architectures and systems based on the knowledge obtained from this paper.**

***Index Terms*—Asynchronous circuits, asynchronous communication, design automation, design methodology, image recognition, logic design, low-power electronics, neural networks, neural network hardware, neuromorphics, parallel architectures, real-time systems, synchronous circuits, very large-scale integration.**

Manuscript received February 8, 2015; revised May 22, 2015; accepted July 21, 2015. Date of publication August 28, 2015; date of current version September 17, 2015. This work was supported by Defense Advanced Research Projects Agency under Contract HR0011-09-C-0002. This paper was recommended by Associate Editor C. J. Alpert.

F. Akopyan, J. Sawada, A. Cassidy, R. Alvarez-Icaza, J. Arthur, P. Merolla, N. Imam, P. Datta, B. Taba, W. P. Risk, B. Jackson, and D. S. Modha are with IBM Research—Almaden, San Jose, CA 95120 USA (e-mail: akopyan@us.ibm.com; sawada@us.ibm.com; andrewca@us.ibm.com; arodrigo@us.ibm.com; arthurjo@us.ibm.com; pameroll@us.ibm.com; ni49@us.ibm.com; pdatta@us.ibm.com; btaba@us.ibm.com; risk@us.ibm.com; bryanlj@us.ibm.com; dmodha@us.ibm.com).

Y. Nakamura is with IBM Research–Tokyo, Tokyo 600-8028, Japan (e-mail: xyutaka@jp.ibm.com).

G.-J. Nam and J. B. Kuang are with IBM Research—Austin, Austin, TX, USA (e-mail: gnam@us.ibm.com; kuang@us.ibm.com).

M. Beakes and B. Brezzo are with the IBM's T. J. Watson Research Center, Yorktown Heights, NY, USA (e-mail: beakes@us.ibm.com; brezzo@us.ibm.com).

R. Manohar is with Cornell University, Ithaca, NY 14850 USA (e-mail: rajit@csl.cornell.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCAD.2015.2474396

I. Introduction

THE REMARKABLE brain, structured with 100 billion parallel computational units (neurons) closely coupled to local memory (1000–10 000 synapses per neuron) and high-fanout event-driven communication, attains exceptional energy efficiency (consuming only 20 W), while achieving unparalleled performance on perceptual and cognitive tasks. The brain's performance arises from its massive parallelism, operating at low-frequency (10 Hz average firing rate) and a low-power density of 10 mW/cm^2, in contrast to contemporary multi-Gigahertz processors with a power density of 100 W/cm^2. The TrueNorth architecture [1], depicted in Fig. 1, is the result of approximating the structure and form of organic neuro-biology within the constraints of inorganic silicon technology. It is a platform for low-power, real-time execution of large-scale neural networks, such as state-of-the-art speech and visual object recognition algorithms [2].

The TrueNorth chip, the latest achievement of the Defense Advanced Research Projects Agency (DARPA) SyNAPSE project, is composed of 4096 neurosynaptic cores tiled in a 2-D array, containing an aggregate of 1 million neurons and 256 million synapses. It attains a peak computational performance of 58 giga-synaptic operations per second (GSOPS) and computational energy efficiency of 400 GSOPS per Watt (GSOPS/W) [3]. We have deployed the TrueNorth chip in 1, 4, and 16-chip systems, as well as on a compact 2"×5" board for mobile applications. The

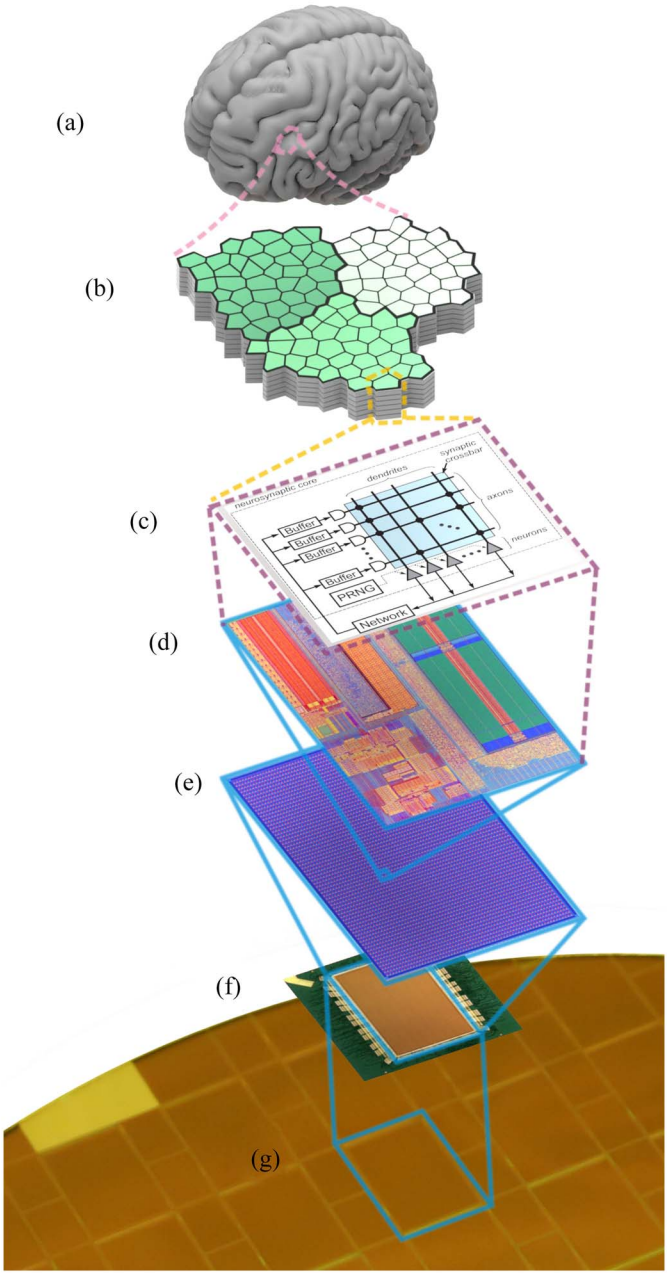

Fig. 1. TrueNorth architecture is inspired by the structure and function of the (a) human brain and (b) cortical column, a small region of densely interconnected and functionally related neurons that form the canonical unit of the brain. Analogously, the (c) neurosynaptic core is the basic building block of the TrueNorth architecture, containing 256 input axons, 256 neurons, and a 64k synaptic crossbar. Composed of tightly coupled computation (neurons), memory (synapses), and communication (axons and dendrites), one core occupies 240×390 μm of (d) silicon. Tiling 4096 neurosynaptic cores in a 2-D array forms a (e) TrueNorth chip, which occupies 4.3 cm^2 in a (f) 28 nm low-power CMOS process and consumes merely 65 mW of power while running a typical computer vision application. The end result is an efficient approximation of cortical structure within the constraints of an (g) inorganic silicon substrate.

TrueNorth native Corelet language, the Corelet programming environment (CPE) [4], and an ever-expanding library of composable algorithms [5] are used to develop applications for the TrueNorth architecture. Prior project achievements include the largest long-distance wiring diagram of the primate brain [6], which informed our architectural choices, as well as a series of simulations using the scalable Compass software simulator, which implements the TrueNorth logical architecture. These simulations scaled from "mouse-scale" and "rat-scale" on Blue Gene/L [7] to "cat-scale" on Blue Gene/P [8] to "human-scale" on LLNL's Sequoia 96-rack Blue Gene/Q [9], [10].

Inspired by the brain's structure, we posited the TrueNorth architecture based on the following seven principles. Each principle led to specific design innovations in order to efficiently reduce the TrueNorth architecture to silicon.

1) *Minimizing Active Power:* Based on the sparsity of relevant perceptual information in time and space, TrueNorth is an event-driven architecture using asynchronous circuits as well as techniques to make synchronous circuits event-driven. We eliminated power-hungry global clock networks, collocated memory and computation (minimizing the distance data travels), and implemented sparse memory access patterns.
2) *Minimizing Static Power:* The TrueNorth chip is implemented in a low-power manufacturing process and is amenable to voltage scaling.
3) *Maximizing Parallelism:* To achieve high performance through 4096 parallel cores, we minimized core area by using synchronous circuits for computation and by time-division multiplexing the neuron computation, sharing one physical circuit to compute the state of 256 neurons.
4) *Real-Time Operation:* The TrueNorth architecture uses hierarchical communication, with a high-fanout crossbar for local communication and a network-on-chip for long-distance communication, and global system synchronization to ensure real-time operation. We define real-time as evaluating every neuron once each millisecond, delineated by a 1 kHz synchronization signal.
5) *Scalability:* We achieve scalability by tiling cores within a chip, using peripheral circuits to tile chips, and locally generating core execution signals (eliminating the global clock skew challenge).
6) *Defect Tolerance:* Our implementation includes circuit-level (memory) redundancies, as well as disabling and routing around faulty cores at the architecture-level to provide robustness to manufacturing defects.
7) *Hardware–Software One-to-One Equivalence:* A fully digital implementation and deterministic global system synchronization enable this contract between hardware and software. These innovations accelerated testing and verification of the chip (both pre- and post-tapeout), as well as enhance programmability, such that identical programs can be run on the chip and simulator.

Realizing these architectural innovations and complex unconventional event-driven circuit primitives in the 5.4 billion transistor TrueNorth chip required new, nonstandard design approaches and tools. Our main contribution, in this paper, is a novel hybrid asynchronous–synchronous flow to interconnect elements from both asynchronous and synchronous design approaches, plus tools to support the design and verification. The flow also includes techniques to operate synchronous circuits in an event-driven manner. We expect this type of asynchronous–synchronous tool flow to be widely used in the

future, not only for neurosynaptic architectures but also for event-driven, power efficient designs for mobile and sensory applications.

To implement efficient event-driven communication circuits, we used fully custom asynchronous techniques and tools. In computational blocks, for rapid design and flexibility, we used a more conventional synchronous synthesis approach. To mitigate testing complexity, we designed the TrueNorth chip to ensure that the behavior of the chip exactly matches a software simulator [9], spike to spike, using a global synchronization trigger. For deployment, we adapted existing very large-scale integration (VLSI) placement tools in order to map logical neural networks to physical core locations on the TrueNorth chip for efficient run-time operation. The end result of this unconventional architecture and nonstandard design methodology is a revolutionary chip that consumes orders of magnitude less energy than conventional processors which use standard design flows.

In this paper, we present the design of the TrueNorth chip and the novel asynchronous–synchronous design tool flow. In Section II, we review related neuromorphic chips and architectures. In Section III, we give an overview of the TrueNorth architecture. In Section IV, we describe our mixed asynchronous–synchronous design approach. In Section V, we provide the chip design details, including the neurosynaptic core and chip periphery, as well as their major components. In Section VI, we discuss our design methodology, focusing on interfaces, simulation, synthesis, and verification tools. In Section VII, we report measured results from the chip. In Section VIII, we present TrueNorth platforms that we built and are currently using. In Section IX, we demonstrate some example applications. In Section X, we describe tools for mapping neural networks to cores. In Sections XI and XII, we propose future system scaling and present the conclusion.

II. Related Work

Focusing on designs that have demonstrated working silicon [11], here are some recent neuromorphic efforts.

The SpiNNaker project at the University of Manchester is a microprocessor-based system optimized for real-time spiking neural networks, where the aim is to improve the performance of software simulations [12]. SpiNNaker uses a custom chip that integrates 18 microprocessors in 102 mm^2 using a 130 nm process. The architecture, being fully digital, uses an asynchronous message passing network (2-D torus) for interchip communication. Using a 48-chip board, SpiNNaker has demonstrated networks with hundreds of thousands of neurons and tens of millions of synapses, and consumes 25–36 W running at real-time [13].

The Neurogrid project at Stanford University is a mixed analog–digital neuromorphic system, where the aim is to emulate continuous-time neural dynamics resulting from the interactions among biophysical components [14]. Neurogrid is a 16-chip system organized in a tree routing network, where neurons on any chip asynchronously send messages to synapses on other chips via multicast communication [15] and a local analog diffuser network. Long-range connections are implemented using a field-programmable gate array (FPGA) daughterboard. Each chip in the system is 168 mm^2 in a 180 nm process, and has a 2D array of 64K analog neurons. Neurons on the same chip share the same parameters (for example, modeling a specific cell type in a layer of cortex). In total, Neurogrid simulates one million neurons in real-time for approximately 3.1 W (total system power including the FPGA).

The BrainScaleS project at the University of Heidelberg is a mixed analog–digital design, where the goal is to accelerate the search for interesting networks by running 1000–10 000 times faster than real-time [16]. BrainScaleS is a wafer-scale system (20 cm diameter in 180 nm technology) that has 200 thousand analog neurons and 40 million addressable synapses, and is projected to consume roughly 1 kW. The architecture is based on a network of high input count analog neural network modules, each with 512 neurons and 128K synapses in 50 mm^2. The communication network at the wafer scale is asynchronous, and connects to a high performance FPGA system using a hierarchical packet-based approach.

Other mixed analog–digital neuromorphic approaches include IFAT from University of California at San Diego [17], and recent work from Institute for Neuroinformatics [18]. Other work has taken a fully synchronous digital approach [19]. See [11] for one comparative view of recent neuromorphic chip architectures.

To the best of the author's knowledge, no previous work has demonstrated a neuromorphic chip in working silicon at a comparable scale to the TrueNorth chip (1 million neurons and 256 million synapses) with comparable power efficiency (65 mW power consumption). This differentiation is realized by our circuit design emphasis on minimizing active/static power, and compact physical design for increasing parallelism. Also hardware–software one-to-one equivalence is a major advantage for testing and application development, a feature that is infeasible for systems based on analog circuits.

III. TrueNorth Architecture

The TrueNorth architecture is a radical departure from the conventional von Neumann architecture. Unlike von Neumann machines, we do not use sequential programs that map instructions into a linear memory. The TrueNorth architecture implements spiking neurons coupled together by the network connecting them. We program the chip by specifying the behavior of the neurons and the connectivity between them. Neurons communicate with each other by sending spikes. The communicated data may be encoded using the frequency, time, and spatial distribution of spikes.

We designed the TrueNorth architecture to be extremely parallel, event-driven, low-power, and scalable, using a neurosynaptic core, as the basic building block of the architecture. The left side of Fig. 2 shows a bipartite graph of neurons, which is a small section of a larger neural network. A TrueNorth core, shown on the right side, is a hardware representation of this bipartite graph of neurons, with arbitrary connectivity between the input and output layers of the graph. A neurosynaptic core contains both computing elements, neurons, for computing the membrane potential using various neuron models, and memory to store neuron connectivity and parameters. By physically locating the memory and the computation close to each other, we localize the data movement,

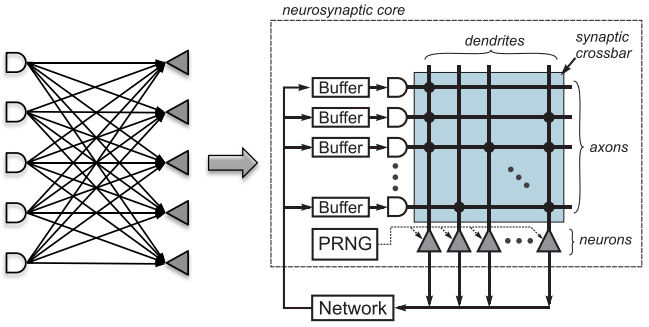

Fig. 2. Bipartite graph of a neural network (left), with arbitrary connections between axons and neurons, and the corresponding logical representation of a TrueNorth core (right).

increasing efficiency, and reducing the power consumption of the chip. By tiling 4096 neurosynaptic cores on a TrueNorth chip, we scale up to a highly-parallel architecture, where each core implements 256 neurons and 64k synapses.

As shown in Fig. 2, a single neurosynaptic core consists of input buffers that receive spikes from the network, axons which are represented by horizontal lines, dendrites which are represented by vertical lines, and neurons (represented by triangles) that send spikes into the network. A connection between an axon and a dendrite is a synapse, represented by a black dot. The synapses for each core are organized into a synaptic crossbar. The output of each neuron is connected to the input buffer of the axon that it communicates with. This axon may be located in the same core as the communicating neuron or in a different core, in which case the communication occurs over the routing network.

The computation of a neurosynaptic core proceeds according to the following steps.

1) A neurosynaptic core receives spikes from the network and stores them in the input buffers.
2) When a 1 kHz synchronization trigger signal called a tick arrives, the spikes for the current tick are read from the input buffers, and distributed across the corresponding horizontal axons.
3) Where there is a synaptic connection between a horizontal axon and a vertical dendrite, the spike from the axon is delivered to the neuron through the dendrite.
4) Each neuron integrates its incoming spikes and updates its membrane potential.
5) When all spikes are integrated in a neuron, the leak value is subtracted from the membrane potential.
6) If the updated membrane potential exceeds the threshold, a spike is generated and sent into the network.

All the computation must finish in the current tick, which spans 1 ms. Although the order of spikes arriving from the network may vary due to network delays, core computation is deterministic within a tick due to the input buffers. In this sense, the TrueNorth chip matches one-to-one with the software simulator, which is one of the advantages of using a fully digital neuron implementation.

The membrane potential $V_j(t)$ of the jth neuron at tick t is computed according to the following simplified equation:

$$V_j(t) = V_j(t-1) + \sum_{i=0}^{255} A_i(t) w_{i,j} s_j^{G_i} - \lambda_j.$$

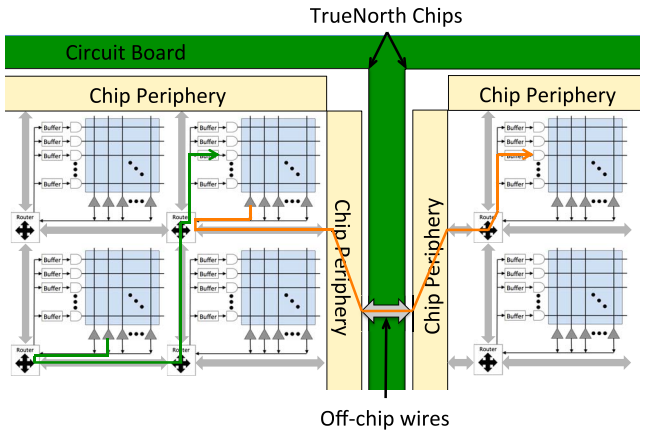

Fig. 3. Corners of two TrueNorth chips, with intrachip (green arrow) and interchip (orange arrow) spike traces.

Here, $A_i(t)$ is 1 if there is an incoming spike (stored temporarily in the buffer) on the ith axon at time t, and $w_{i,j}$ is 1 if the ith axon is connected to the jth dendrite. Otherwise, the values are 0. We assign one of four types to each axon, and designate the type of the ith axon with G_i, an integer from 1 to 4. The synaptic weight between any axon of type G and the jth dendrite is given by the integer value s_j^G. The integer value λ_j is a leak, subtracted from the membrane potential at each tick. When the updated membrane potential $V_j(t)$ is larger than the threshold value α_j, the jth neuron generates a spike and injects it into the network, and the state $V_j(t)$ is reset to R_j. The actual neuron equations implemented in the TrueNorth chip are more sophisticated [20]. For example, we support: stochastic spike integration, leak, and threshold using a pseudo random number generator (PRNG); programmable leak signs depending on the sign of the membrane potential; and programmable reset modes for the membrane potential after spiking. The axon and synaptic weight constraints are most effectively handled by incorporating the constraints directly in the learning procedures (for example, back propagation). We find that training algorithms are capable of finding excellent solutions using the free parameters available, given the architectural constraints.

We created the TrueNorth architecture by connecting a large number of neurosynaptic cores into a 2-D mesh network, creating an efficient, highly-parallel, and scalable architecture. The TrueNorth architecture can be tiled not only at the core-level but also at the chip-level. Fig. 3 demonstrates spike communication within one chip, as well as between two adjacent chips. The green arrow shows a trace of an intrachip spike traveling from the lower-left core to the upper-right core using the on-chip routing network. To extend the 2-D mesh network of cores beyond the chip boundary, we equipped the TrueNorth chip with custom peripheral circuitry that allows seamless and direct interchip communication (demonstrated by the orange arrow in Fig. 3). As expected, the time and energy required to communicate between cores physically located on different chips is much higher than to communicate between cores on the same chip. We will revisit this issue in Section X.

To implement this architecture in a low-power chip, we chose to use event-driven computation and communication.

This decision led us to using a mixed asynchronous–synchronous approach, as discussed in the next section.

IV. Mixed Synchronous–Asynchronous Design

The majority of typical semiconductor chips only utilize the synchronous design style due to a rapid design cycle and the availability of computer-aided design (CAD) tools. In this approach, all state holding elements of the design update their states simultaneously upon arrival of the global clock edge. In order to assure correct operation of synchronous circuits, a uniform clock distribution network has to be carefully designed to guarantee proper synchronization of circuit blocks throughout the entire chip. Such clock distribution networks have penalties in terms of power consumption and area. Global clock trees operate continuously with high slew rates to minimize the clock skew between clocked sequential elements, and consume dynamic power even when no useful task is performed by the circuits. As a result, if used in an activity dependent design such as our neurosynaptic architecture, these clock trees would waste significant energy. Clock gating is a technique commonly used to mitigate the power drawbacks of global clock trees. However, clock gating is generally performed at a very coarse level and requires complex control circuitry to ensure proper operation of all circuit blocks. Another option for synchronous designs is to distribute slow-switching clocks across the chip and use phase-locked loops (PLLs) to locally multiply the slow clocks at the destination blocks, but PLLs have area penalties and are not natively activity-dependent.

In contrast, asynchronous circuits operate without a clock, by using request/acknowledge handshaking to transmit data. The data-driven nature of asynchronous circuits allows a circuit to be idle without switching activity when there is no work to be done. In addition, asynchronous circuits are capable of correct operation in the presence of continuous and dynamic changes in delays [21]. Sources of local delay variations may include temperature, supply voltage fluctuations, process variations, noise, radiation, and other transient phenomena. As a result, the asynchronous design style also enables correct circuit operation at lower supply voltages.

For the TrueNorth chip, we chose to use a mixed asynchronous–synchronous approach. For all the communication and control circuits, we chose the asynchronous design style, while for computation, we chose the synchronous design style. Since TrueNorth cores operate in parallel, and are governed by spike generation/transmission/delivery (referred to as events), it is natural to implement all the routing mechanisms asynchronously. The asynchronous control circuitry inside each TrueNorth core ensures that the core is active only when it is necessary to integrate synaptic inputs and to update membrane potentials. Consequently, in our design there is no need for a high-speed global clock, and communication occurs by means of handshake protocols.

As for computational circuits, we used the synchronous design style, since this method aids in implementing complex neuron equations efficiently in a small silicon area, minimizing the leakage power. However, the clock signals for the synchronous circuit blocks are generated locally in each core by an asynchronous control circuit. This approach generates a clock pulse only when there is computation to be performed, minimizing the number of clock transitions and ensuring the lowest dynamic power consumption possible.

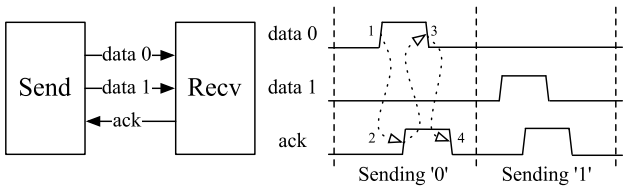

Fig. 4. Asynchronous four-phase handshake.

For the asynchronous circuit design, we selected a quasi delay-insensitive (QDI) circuit family. The benefits, as well as the limitations of this asynchronous style have been thoroughly analyzed in [21]. The exception to the QDI design style was for off-chip communication circuits, where we used bundled-data asynchronous circuits to minimize the number of interface signals. The QDI design style, based on Martin's synthesis procedure [22], decomposes communicating hardware processes (CHP) into many fine-grained circuit elements operating in parallel. These processes synchronize by communicating tokens over delay-insensitive channels that are implemented using a four-phase handshake protocol. One of the main primitives of such communication is a Muller C-element [23]. A single-bit communication protocol is shown in Fig. 4. Here, the sending process asserts the "data 0" line to communicate a "0" value, shown by the rising transition (1), which the receiving process then acknowledges with a rising transition (2). Subsequently, the channel is reset with falling transitions (3,4) to a neutral state. As seen in Fig. 4, sending a "1" value, demonstrated on the next communication iteration, is quite similar. The dotted arrows in Fig. 4 indicate the causality of the handshake operations.

To implement QDI asynchronous pipelines we used precharge half-buffers, and precharge full-buffers for computational operations based on their robust performance in terms latency and throughput, even with complex logic. As for simple FIFO-type buffers and control-data slack matching, we used weak-condition half-buffers, due to their short cycle time (ten transitions per cycle) and small size in terms of transistor count. These circuit primitives are described in detail by Lines [24].

This novel mixed asynchronous–synchronous design approach enables us to significantly reduce the active and static power, by making the circuits event-driven, and tuning them for low-power operation. Nonetheless, the circuits operate at a high speed, making it possible to run the chip at real-time. For each block, depending on the requirements, we have used either synchronous or asynchronous design style, as discussed in the next section.

V. TrueNorth Chip Design

A. High-Level Chip Description

In this section, we discuss the detailed design of the TrueNorth chip, implemented based on the presented

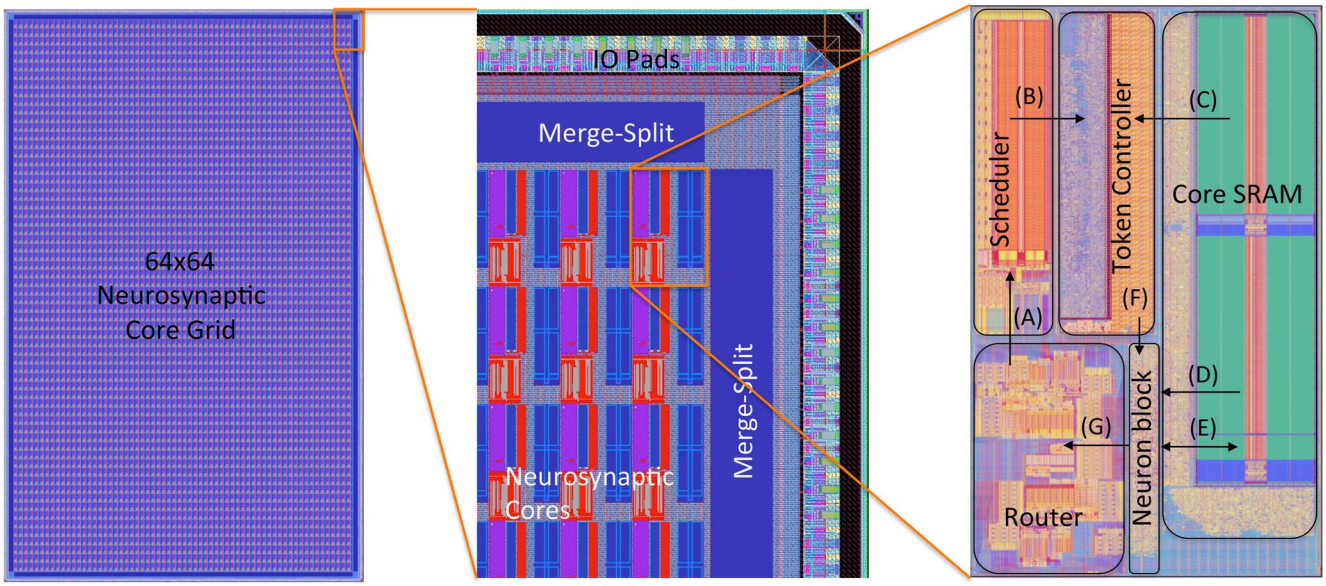

Fig. 5. TrueNorth chip (left) consists of a 2-D grid of 64 × 64 neurosynaptic cores. At the edge of the chip (middle), the core grid interfaces with the merge-split block and then with I/O pads. The physical layout of a single core (right) is overlaid with a core block diagram. See text for details on the data flow through the core blocks. The router, scheduler, and token controller are asynchronous blocks, while the neuron and static random-access memory (SRAM) controller blocks are synchronous.

architecture and the previously discussed circuit styles. We first explain the overall chip design, introducing all the TrueNorth blocks in this section, and then describe each individual block in the following sections.

Fig. 5 shows the physical layout of a TrueNorth chip. The full chip consists of a 64 × 64 array of neurosynaptic cores with associated peripheral logic. A single core consists of the scheduler block, the token controller block, the core SRAM, the neuron block, and the router block.

The router communicates with its own core and the four neighboring routers in the east, west, north, and south directions, creating a 2-D mesh network. Each spike packet carries a relative *dx*, *dy* address of the destination core, a destination axon index, a destination tick at which the spike is to be integrated, and several flags for debugging purposes. When a spike arrives at the router of the destination core, the router passes it to the scheduler, shown as (A) in Fig. 5.

The main purpose of the scheduler is to store input spikes in a queue until the specified destination tick, given in the spike packet. The scheduler stores each spike as a binary value in an SRAM of 16 × 256-bit entries, corresponding to 16 ticks and 256 axons. When a neurosynaptic core receives a tick, the scheduler reads all the spikes $A_i(t)$ for the current tick, and sends them to the token controller (B).

The token controller controls the sequence of computations carried out by a neurosynaptic core. After receiving spikes from the scheduler, it processes 256 neurons one by one. Using the membrane potential equation introduced in Section III: for the *j*th neuron, the synaptic connectivity $w_{i,j}$ is sent from the core SRAM to the token controller (C), while the rest of the neuron parameters (D) and the current membrane potential (E) are sent to the neuron block. If the bit-wise AND of the synaptic connectivity $w_{i,j}$ and the input spike $A_i(t)$ is 1, the token controller sends a clock signal and an instruction (F) indicating axon type G_i to the neuron block, which in turn adjusts the membrane potential based on $s_j^{G_i}$. In other words, the neuron block receives an instruction and a clock pulse only when two conditions are true: there is an active input spike and the associated synapse is active. Thereby, the neuron block computes in an event-driven fashion.

When the token controller completes integrating all the spikes for a neuron, it sends several additional instructions to the neuron block: one for subtracting the leak λ_i, and another for checking the spiking condition. If the resulting membrane potential $V_i(t)$ is above the programmable threshold, the neuron block generates a spike packet (G) for the router. The router in turn injects the new spike packet into the network. Finally, the updated membrane potential value (E) is written back to the core SRAM to complete the processing of a single neuron.

When a core processes all the neurons for the current tick, the token controller stops sending clock pulses to the neuron block, halting the computation. The token controller then instructs the scheduler to advance its time pointer to the next tick. At this point, besides delivering incoming spikes to the scheduler, the neurosynaptic core goes silent, waiting for the next tick.

The chip's peripheral interfaces allow the extension of the 2-D grid of neurosynaptic cores beyond the chip boundaries. Since the quantity of available I/O pins is limited, the merge-split blocks at the edges of the chip merge spikes coming from multiple buses into a single stream of spikes going out of the chip. Conversely, this peripheral unit also distributes a stream of incoming off-chip spikes to multiple buses in the core array.

Sections V-B–V-H introduce the blocks constituting the TrueNorth chip. We first introduce the blocks that reside in each TrueNorth neurosynaptic core, and then move to the chip's peripheral circuitry.

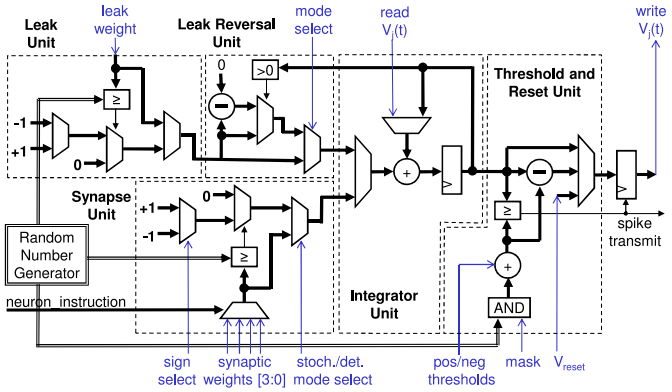

Fig. 6. Neuron block diagram.

B. *TrueNorth Core Internals: Neuron Block*

The neuron block is the TrueNorth core's main computational element. We implemented a dual stochastic and deterministic neuron based on an augmented integrate-and-fire neuron model [20]. Striking a balance, the implementation complexity is less than the Hodgkin–Huxley or Izhikevich neuron models, however, by combining 1–3 simple neurons, we are able to replicate all 20 of the biologically observed spiking neuron behaviors cataloged by Izhikevich [25]. The physical neuron block uses time-division multiplexing to compute the states of 256 logical neurons for a core with a single computational circuit. In order to simplify implementation of the complex arithmetic and logical operations, the neuron block was implemented in synchronous logic, using a standard application-specified integrated circuit (ASIC) design flow. However, it is event-driven, because the token controller only sends the exact number of clock pulses required for the neuron block to complete its computation, as described in Section V-F.

The block diagram shown in Fig. 6 depicts the five major elements of the neuron block, with input/output parameters from/to the core SRAM shown in blue. The synaptic input unit implements stochastic and deterministic inputs for four different weight types with positive or negative values $s_j^{G_i}$. The leak and leak reversal units provide a constant bias (stochastic or deterministic) on the dynamics of the neural computation. The integrator unit sums the membrane potential from the previous tick with the synaptic inputs and the leak input. The threshold and reset unit compares the membrane potential value with the threshold value. If the membrane potential value is greater than or equal to the threshold value, the neuron block resets the membrane potential and transmits a spike event. The random number generator is used for the stochastic leak, synapse, and threshold functions. The core SRAM, external to this block, stores the synaptic connectivity and weight values, the leak value, the threshold value, the configuration parameters, as well as the membrane potential, $V_j(t)$. At the beginning and end of a neural computation cycle, the membrane potential is loaded from and then written back to the core SRAM.

As previously mentioned, even though the neuron block was implemented using synchronous design flow to take advantage of rapid and flexible design cycles, it is fully event-driven to minimize the active power consumption. Additionally, multiplexing a single neuron circuit for multiple logical neurons reduced the area of the block, while increasing the parallelism and reducing the static power consumption.

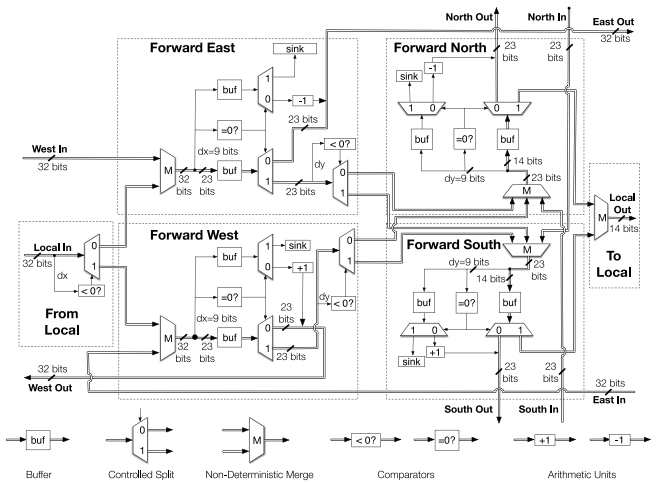

Fig. 7. Internal structure of the router (top) composed from five QDI asynchronous circuit blocks (bottom).

C. *TrueNorth Core Internals: Router*

The TrueNorth architecture supports complex interconnected neural networks by building an on-chip network of neurosynaptic cores. The 2-D mesh communication network, which is responsible for routing spike events from neurons to axons with low latency, is a graph where each node is a core and the edges are formed by connecting the routers of nearest-neighbor cores together. Connections between cores, where neurons on any module individually connect to axons on any module, are implemented virtually by sending spike events through the network of routers.

As illustrated in Fig. 7, each router is decomposed into six individual processes: 1) from local; 2) forward east; 3) forward west; 4) forward north; 5) forward south; and 6) to local. There are five input ports—one that receives the routing packets from spiking neurons within the local core and four others that receive routing packets from nearest-neighbor routers in the east, west, north, and south directions of the 2-D mesh. Upon receiving a packet, the router uses the information encoded in the *dx* (number of hops in the *x* direction as a 9-bit signed integer) or *dy* (number of hops in the *y* direction as a 9-bit signed integer as well) fields to send the packet out to one of five destinations: the scheduler of the local core, or nearest neighbor routers to the east, west, north, or south directions. Thus, a spiking neuron from any core in the network can connect to an axon of any destination core within a 9-bit routing window by traversing the 2-D mesh network through local routing hops. If a spike needs to travel further than this routing window (up to 4 chips in either direction), we use repeater cores.

Packets are routed first in the horizontal direction and the *dx* field is decremented (eastward hop, $dx > 0$) or incremented (westward hop, $dx < 0$) with each routing hop. When *dx*

becomes 0, the associated bits are stripped from the packet and routing in the vertical direction initiates. With each vertical hop, dy is decremented (northward hop, $dy > 0$) or incremented (southward hop, $dy < 0$). When dy becomes 0, the associated bits are stripped from the packet and the remaining 14 bits are sent to the scheduler of the local core. By prioritizing routing a packet in the east/west directions over the north/south directions, network deadlock through dependency cycles among routers is precluded.

Because more than one message can contend for the same router, its resources are allocated on a first-come first-serve basis (simultaneously arriving packets are arbitrated) until all requests are serviced. For any spike packet that has to wait for a resource, it is buffered and backpressure is applied to the input channel, guaranteeing that no spikes will be dropped. Depending on congestion, this backpressure may propagate all the way back to the generating core, which stalls until its spike can be injected into the network. As a result, the communication time between a spiking neuron and a destination axon will fluctuate due to on-going traffic in the spike-routing network. However, we hide this network nondeterminism at the destination by ensuring that the spike delivery tick of each spike is configured to be longer than the maximum communication latency between its corresponding source and destination. In order to mitigate congestion, each router has built-in buffering on all input and output channels in order to service more spikes at a higher throughput. The router can route several spikes simultaneously, if their directions are not interfering.

The router was constructed with fully asynchronous QDI logic blocks depicted at the bottom of Fig. 7, and described as follows.

1) A buffer block enables slack matching of internal signals to increase router throughput.
2) A controlled split block routes its input data to one of two outputs depending on the value of a control signal.
3) A nondeterministic merge block routes data from one of two inputs to one output on a first-come first-serve basis. An arbiter inside the block resolves metastabilities arising from simultaneous arrival of both inputs.
4) Comparator blocks check the values of the dx and dy fields of the packet to determine the routing direction.
5) Arithmetic blocks increment or decrement the value of the dx and dy fields.

The implemented asynchronous design naturally enables high-speed spike communication during times of (and in regions of) bursty neural activity, while maintaining low-power consumption during times of (and in regions of) sparse activity. With the mesh network constructed from the routers, the TrueNorth chip implements hierarchical communication; a spike is sent globally through the mesh network using a single packet, which fans out to multiple neurons locally at the destination core. This reduces the network traffic, enabling the TrueNorth chip to operate in real-time.

D. TrueNorth Core Internals: Scheduler

Once the spike packet arrives at the router of the destination core, it is forwarded to the core's scheduler. The scheduler's function is to deliver the incoming spike to the right axon at the proper tick. The spike packet at the scheduler input has the following format (the dx and dy destination core coordinates have been stripped by the routing network prior to the arrival of the packet to the scheduler).

delivery tick	destination axon index	debug bits
4 bits	8 bits	2 bits

Here, delivery tick represents the least significant 4-bit of the tick counter; destination axon index points to one of 256 axons in the core; debug bits are used for testing and debugging purposes. The delivery tick of an incoming spike needs to account for travel distance and maximum network congestion, and must be within the next 15 ticks after the spike generation (represented by 4 bits).

1) Scheduler Operation: The scheduler contains spike write/read/clear control logic and a dedicated 12-transistor (12T) 16 × 256-bit SRAM. The 256 bitlines of the SRAM correspond to the 256 axons of the core, while 16 wordlines correspond to the 16 delivery ticks. The overall structure of the scheduler is outlined in Fig. 8.

The scheduler performs a WRITE operation to the SRAM when it receives a spike packet from the router. The scheduler decodes an incoming packet to determine when (tick) and where (axon) a spike should be delivered. It then writes a value 1 to the SRAM bitcell corresponding to the intersection of the destination axon (SRAMs bitline) and the delivery tick (SRAMs wordline). All other SRAM bits remain unchanged. This requires a special SRAM that allows a WRITE operation to a single bit. The READ and CLEAR operations to the SRAM are initiated by the token controller. At the beginning of each tick, the token controller sends a request to the scheduler to read all the spikes for the current tick (corresponding to an entire wordline of the SRAM); these spikes are sent to the token controller. After all spikes are integrated for all 256 neurons in the core, the token controller requests the scheduler to clear the wordline of the SRAM corresponding to the current tick. Since the incoming spikes may arrive from the router at any time, the scheduler is designed to perform READ/CLEAR operations in parallel with a WRITE operation, as long as these operations are targeting different wordlines.

2) Scheduler SRAM: The SRAM used in the scheduler has a special interface to store spikes that are coming from the router (one at a time), while in parallel reading and clearing spikes corresponding to the current tick for the entire 256 axon array. The WRITE operation is performed by asserting the write wordline and bitline. The bitcell at the intersection of these lines will be updated with a value 1, while the rest of the SRAM is unchanged. The READ operation is initiated by asserting one of 16 read ports; this operation reads the entire 256 bits of the corresponding wordline. Similarly, the CLEAR operation is performed by asserting one of the 16 clear ports, which sets all the bitcells in the corresponding wordline to the value 0. The details of the physical implementation for the scheduler SRAM are described in Section V-E.

3) Scheduler Decoders: The asynchronous decoders of the scheduler were implemented using 256 C-elements on the

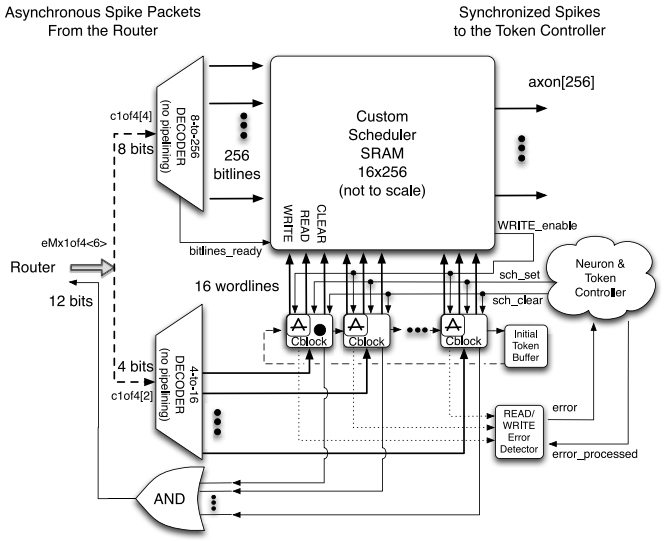

Fig. 8. Overall scheduler structure. Note that the scheduler SRAM is rotated 90° with respect to the bitcell orientation shown in Fig. 9.

bitline side and 16 C-elements on the wordline side. From the spike packet, the bitline decoder receives the axon number (represented in 8 bits by a 1-of-4 coding scheme), and asserts one of the 256 bitlines. Similarly, the wordline decoder receives the delivery tick of a spike (represented in 4 bits by a 1-of-4 coding scheme) and asserts one of the 16 wordlines using custom self-timed control blocks. The decoders process incoming spikes one at a time to avoid race conditions. A 16-bit AND-tree combines ready signals from the control blocks, enabling the router to forward the next incoming spike. The ready signals prevent the router from sending the next spike until the previous one is successfully written to the SRAM.

4) Scheduler Control Blocks: The scheduler control blocks (Cblocks) asynchronously regulate the READ, WRITE, and CLEAR operations to the scheduler SRAM. Cblocks receive incoming spike requests directly from the wordline decoder in a one-hot manner. The Cblock that receives a value 1 from the decoder initiates a WRITE to the corresponding wordline, as soon as the SRAM is ready (WRITE_enable = 1). Only one Cblock is allowed to perform READ and CLEAR operations in a given tick. The privilege of performing these operations is passed sequentially to the subsequent Cblock at the end of every tick using an asynchronous token. When the token controller asserts sch_set or sch_clear, the Cblock that possesses the asynchronous token performs the READ and CLEAR operations, respectively. If a WRITE is requested at the Cblock with the token, the scheduler reports an error to the neuron block, since this would imply a READ-WRITE conflict in the scheduler SRAM. The neuron block acknowledges this scenario with the error_processed signal.

The scheduler's logic was implemented using the asynchronous design style to minimize the dynamic power consumption. The asynchronous approach is beneficial here, since spike packets may arrive from the router at any time with variable rate. The scheduler also serves the function of a synchronizer by aligning spikes and releasing them to axons at the proper tick. This synchronization is critical to maintain the hardware–software one-to-one equivalence by masking the nondeterminism of the routing network, while enabling real-time operation.

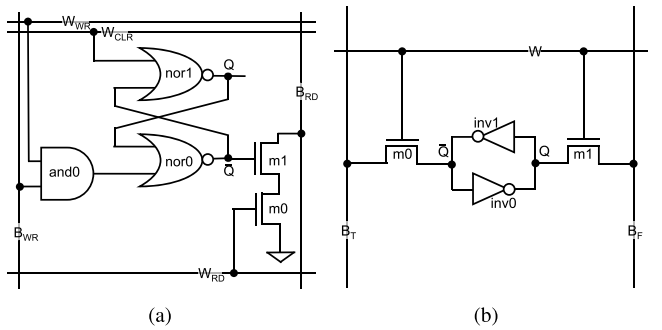

Fig. 9. SRAM bitcells. (a) 12T scheduler SRAM Cell (simplified). (b) 6T core SRAM cell.

E. TrueNorth Core Internals: SRAMs

Each TrueNorth core has two SRAM blocks. One resides in the scheduler and functions as the spike queue storing incoming events for delivery at future ticks. The other is the primary core memory, storing all synaptic connectivity, neuron parameters, membrane potential, and axon/core targets for all neurons in the core.

1) Scheduler SRAM: The scheduler memory uses a fully custom 12T bitcell (see Section V-D for block details and sequencing information). Unlike a standard bitcell, the 12T cell performs three operations, with a port for each: READ, WRITE, and CLEAR. All signals are operated full swing, transitioning fully between GND and VDD, requiring no sense amplifiers. The READ operation fetches the state of a row of cells, requiring a read wordline, W_{RD}, and a read bitline, B_{RD}. The WRITE operation sets one bitcell (selecting both row and column) to 1, requiring a word select, W_{WR}, and a bit select, B_{WR}. The CLEAR operation resets the state of a row of cells to 0, requiring only a clear wordline, W_{CLR}.

The 12T cell is structured similar to a set–reset latch, composed of a pair of cross-coupled NOR gates, which maintain an internal state, Q, and its inverse, $\overline{Q}$ [Fig. 9(a)]. For a CLEAR operation, the W_{CLR} input to nor1 is pulled to 1, driving Q to 0, which drives $\overline{Q}$ to 1. For a WRITE operation, an input to nor0 is pulled to 1, driving $\overline{Q}$ to 0, which drives Q to 1. For this operation, the cell ANDs its row and column selects (W_{WR} and B_{WR}); however, in the physical implementation, we integrate the AND and NOR logic (and0 and nor0) into a single six transistor gate to save area (not shown). For a READ operation, we access a stack of two NFETs. When W_{RD} and $\overline{Q}$ are 1, the stack pulls B_{RD} to 0. When Q is 1 ($\overline{Q}$ is 0), the stack is inactive and does not pull B_{RD} down and it remains 1 (due to a precharge circuit).

2) Core SRAM: The primary core memory uses a 0.152 μm^2 standard 6-transistor (6T) bitcell [26] [Fig. 9(b)]. The standard SRAM cell uses READ and WRITE operations, sharing both wordlines and bitlines. The bitcell consists of a cross-coupled inverter pair and two access transistors. The SRAM module uses a standard sense amplifier scheme, reducing the voltage swing on bitlines during a READ operation and saving energy.

The SRAM is organized into 256 rows by 410 columns (not including redundant rows and columns). Each row corresponds to the information for a single neuron in the core (see Section V-F). The neuron's 410 bits correspond to its synaptic connections, parameters and membrane potential $V_j(t)$, its core and axon targets, and the programmed delivery tick.

synaptic connections	$V_j(t)$ & neuron parameters	spike destination	spike delivery tick
256 bits	124 bits	26 bits	4 bits

Both SRAMs were designed to minimize the power consumption and the area, while meeting the necessary performance requirement. For example, we reduced the size of the FETs in the SRAM drivers, lowering the signal slew rate to 3.5 V/ns in order to save power. We also arranged the physical design floorplan in a way that minimizes the distance between the SRAM and the computational units, reducing the data movement and the dynamic power consumption. Additionally, we used redundant circuit structures to improve the yield, since the majority of the manufacturing defects occur in the SRAMs.

F. TrueNorth Core Internals: Token Controller

Now that we have introduced the majority of the core blocks, the missing piece is their coordination. To that end, the token controller orchestrates which actions occur in a core during each tick. Fig. 10 shows a block diagram of the token controller. At every tick, the token controller requests 256 axon states from the scheduler corresponding to the current tick. Next, it sequentially analyzes 256 neurons. For each neuron, the token controller reads a row of the core SRAM (corresponding to current neuron's connectivity) and combines this data with the axon states from the scheduler. The token controller then transitions any updates into the neuron block, and finally sends any pending spike events to the router.

Algorithmically, the control block performs the following events at each tick.

1) Wait for a tick.
2) Request current axon activity $A_i(t)$ from the scheduler.
3) Initialize READ pointer of the core SRAM to row 0.
4) Repeat for 256 neurons, indexed as j:
   a) read neuron data from the current SRAM row;
   b) repeat for 256 axons, indexed as i:
      i) send compute instructions to the neuron block only if spike $A_i(t)$ and synaptic connection $w_{i,j}$ are both 1; otherwise skip;
   c) apply leak;
   d) check threshold, reset membrane potential if applicable;
   e) send spike to the router, if applicable;
   f) write back the membrane potential to the core SRAM;
   g) increment the SRAM READ pointer to next row.
5) Request to clear current axon activity $A_i(t)$ in the scheduler SRAM.
6) Report any errors.

This algorithm, known as the "dendritic approach," enables storing all the data for a neuron in a single core SRAM row and only reading/writing each row once per tick. The dendritic approach sets a constant bound on the number of RAM operations, decoupling the SRAM read power consumption from the synaptic activity. In contrast, the more common "axonal approach" built into the Golden Gate neurosynaptic core [27], handles every input spike as an event that triggers multiple SRAM operations.

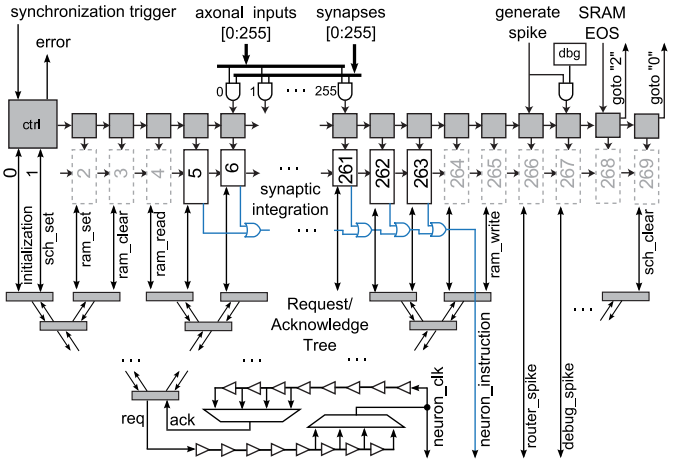

Fig. 10. Token controller overview (asynchronous registers shown as gray squares).

Internally, to process each neuron, the token controller sends a token through a chain of asynchronous shift registers (267 registers) and corresponding logic blocks. In the asynchronous register array, 256 registers contain data about which neurons need to be updated. The remaining registers handle the control logic between the scheduler, the core SRAM, and the router (in case of any spikes). At each stage, a shift register may generate a timed pulse to a given block, send an instruction and generate a clock pulse for the neuron block, or directly pass the token to the next register. Asynchronous registers responsible for executing synaptic integration instructions in the neuron block have an input port that checks that a given synapse is active and that there is a spike on that axon before generating an instruction and a trigger pulse for the associated neuron. If there is no synaptic event, the token advances to the next register, conserving energy and time.

Under extreme spike traffic congestion in the local routing network, the asynchronous token may not traverse the ring of registers within one tick, resulting in a timing violation. In such a scenario, a system-wide error flag is raised by the token controller indicating to the system that spike data may be lost and the computational correctness of the network is not guaranteed at this point, although the chip will continue to operate.

Depending on the application, dropping spike data may or may not be problematic for the system. Sensory information processing systems are designed to be robust at the algorithmic level. The system must be robust to variance in the input data in order to operate in realistic conditions. Similarly, additive noise should not destroy system performance. Thus, while losing data is undesirable, the system performance should degrade gracefully, and not catastrophically. Importantly, however, when data is lost, the errors are flagged by the system to notify the user.

When an error condition is detected, there are several approaches to correcting the situation. First, the user may retry the computation with a longer tick, a useful method when running faster than real-time. For real-time systems, however, the appearance of an error condition generally means that there is a problem in the network that needs to be corrected. This means either: 1) adjusting the I/O bandwidth requirements of the network or 2) changing the placement to reduce the interchip communication (see Section X).

The token controller comprises both asynchronous and synchronous components to coordinate actions of all core blocks. However, the token controller design is fully event-driven with commands and instructions generated to other blocks only when necessary, while keeping the core dormant, if there is no work to be done. The Token controller is a key to lowering active power consumption and implementing the high-speed design required for real-time operation. The token controller description completes the overview of blocks inside of a TrueNorth core.

G. TrueNorth Chip Periphery: Merge-Split Blocks and Serializer/Deserializer Circuits

Now we move from the design of an individual core, to the design of the peripheral circuitry that interfaces the core array to the chip I/O. The core array has 64 bi-directional ports on each of the four edges of the mesh boundary, corresponding to a total of 8320 wires for east and west edges and 6272 wires for north and south.[1] Due to the limited number of I/O pads (a few hundred per side), it is infeasible to connect the ports from the core array directly off chip without some form of time multiplexing.

To implement this multiplexing, we designed a merge-split block that interfaces 32 ports from the core array to the chip I/O, shown in Fig. 11. At the center of the design is the core array that has 256 bidirectional ports at its periphery, corresponding to the edge of the on-chip routing network. Groups of 32 ports interface to a merge-split block which multiplex and demultiplex packets to a single port before connecting to the pad ring. In total, there are eight merge-split blocks around the chip periphery. The merge-split was designed so that adjacent chips can directly interface across chip boundaries, thereby supporting chip tiling. Specifically, spikes leaving the core array are tagged with their row (or column) before being merged onto the time-multiplexed link. Conversely, spikes that enter the chip from a multiplexed link are demultiplexed to the appropriate row (or column) using the tagged information. To maintain a high throughput across chip boundaries, the merge-split blocks implement internal buffering via asynchronous FIFOs.

The merge-split is made from two independent paths: 1) a merge-serializer path that is responsible for sending spikes from the core array to the I/O and 2) a deserializer-split path that is responsible for receiving spikes from the I/O and sending them to the core array. The merge-serializer path operates as follows. First, spikes packets leaving the core array enter a Merge, where they are tagged with an additional 5 bits to indicate which of the 32 ports they entered on. Logically, the Merge is a binary tree built from two-input one-output circuits with arbitration that combine 32 ports to a single port. Finally, this single port enters a Serializer circuit that converts the packet into an efficient bundled-data four-phase protocol suitable for off-chip communication, shown in Fig. 12.

The converse path consists of a deserializer-split. Here, spike packets arrive from the off-chip link using the bundled-data four-phase protocol. These packets are first converted back into a delay-insensitive four-phase protocol via the deserializer circuit. Next, the split block steers the packet to the appropriate row (or column) based on the tagged information that originated from the adjacent chip. Logically, the split is a binary tree built from one-input two-output steering circuits. Note that the row (or column) tag is stripped off before entering the core array.

The router and the merge-split blocks operate in tandem, and we implemented both of them using an asynchronous design style. The QDI approach minimizes routing power consumption when the circuits are idle and maximizes throughput when

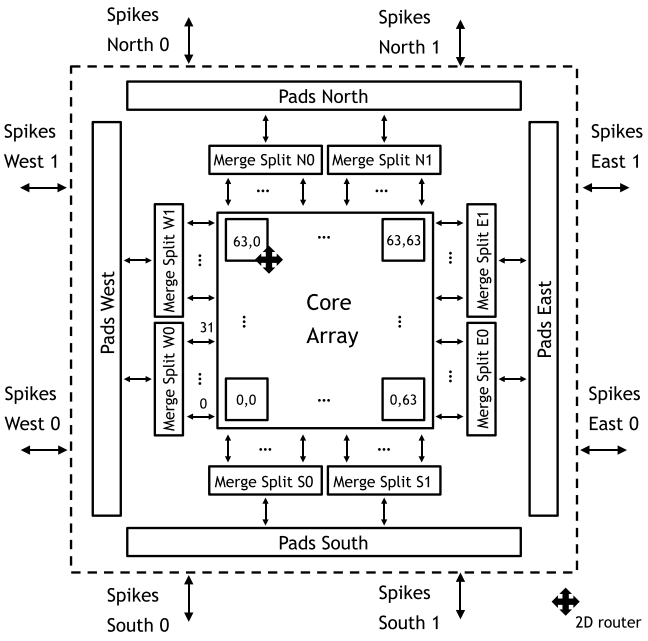

Fig. 11. Top level blocks of the TrueNorth chip architecture.

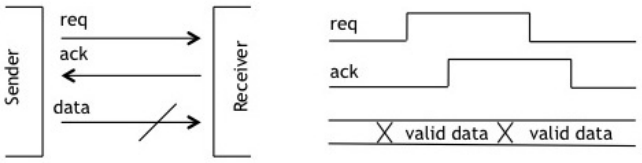

Fig. 12. Two-phase bundled data protocol used for interchip communication. To maximize I/O bandwidth, we send data on both rising and falling edges of the req signal (double data rate), and use bundled data.

[1]Each uni-directional port uses 32- and 24-bit for east–west and north–south edges, respectively. These ports are encoded using 1-in-4 codes plus an acknowledge signal, corresponding to 65 and 49 wires, respectively. The north and south ports require fewer bits since the router data path in the vertical direction drops the *dx* routing information.

the routing structures have to handle many spikes in flight at the same time. When no spikes are in flight, the router and the merge-split blocks remain dormant and consume only a minimal amount of leakage power. The asynchronous design style also allows us to operate at a lower voltage and in the presence of longer switching delays.

By extending the 2-D mesh network beyond the chip boundary, the merge-split blocks enable efficient scaling to multichip systems with tens of millions of neurons. The asynchronous handshake interface of the TrueNorth chip does not require high-speed clocks or any external logic for interchip communication, reducing the power requirement further.

H. TrueNorth Chip Periphery: Scan Chains and I/O

Here, we cover the TrueNorth chip's interface for programming and testing. The TrueNorth chip is programmed through scan chains. Each neurosynaptic core has eight scan chains with a multiplexed scan interface. Three chains are used to read, write, and configure the core SRAM. Four additional chains are utilized to program the parameters of neurosynaptic cores such as axon types and PRNG seed values, as well as to test the synchronous logic. The last chain is a boundary scan isolating the asynchronous circuits and breaking the acknowledge feedback loop to test the asynchronous logic independently from the synchronous logic.

The scan chain runs at a modest speed of 10 MHz, without PLLs or global clock trees on the chip. In order to speed up the programming and testing of the chip, we added several improvements to the scan infrastructure. First, 64 neurosynaptic cores in a single row share a scan interface, reducing the chip I/O pin count used for scanning, while allowing cores in different rows to be programmed in parallel. Second, neurosynaptic cores in the same row may be programmed simultaneously with identical configuration, or sequentially programmed with unique configurations. This technique allows identical test vectors to be scanned into all neurosynaptic cores to run chip testing in parallel. Third, the TrueNorth chip sends out the exclusive-OR of the scan output vectors from neurosynaptic cores selected by the system. This property enables accelerated chip testing by checking the parity of the scan output vectors from multiple cores, while also having the ability of scanning out each individual core, if necessary.

This section completes the discussion of the TrueNorth chip design details. One of our key innovations is the adoption of asynchronous logic for activity-dependent blocks, while using conventional synchronous logic, driven by locally-generated clocks, for computational elements. In the following section, we discuss the custom hybrid design methodology that we developed to implement this state-of-the-art mixed asynchronous–synchronous chip.

VI. Chip Design Methodology

A. TrueNorth Tool Flow

The combination of asynchronous and synchronous circuits in the TrueNorth chip poses a unique challenge in its design methodology. In the design approaches discussed in this section, we show how we combined conventional ASIC and custom design tools to design this chip.

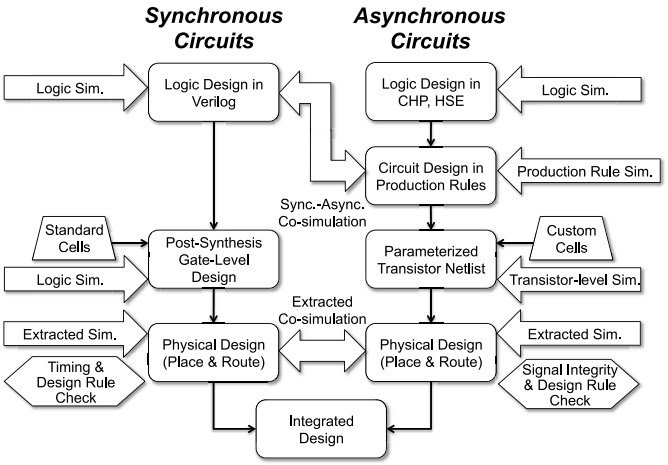

Fig. 13. TrueNorth design tool flow.

The overall design flow is summarized in Fig. 13. The logic design of the synchronous circuits in the TrueNorth chip was specified using the Verilog hardware description language. The synchronous logic goes through the standard synthesis and place and route procedures. However, asynchronous circuit design and verification are not supported by commercial chip design tools. Instead we used a mix of academic tools [28] and tools developed internally. For asynchronous design we used CHP [29] hardware description language and handshaking expansions [30] for high-level description. We then employed asynchronous circuit toolkit (developed at Cornell University, which evolved from CAST [31]) to aid in transforming the high-level logic into silicon, utilizing a transistor stack description format, called production rules. Though several techniques exist for automated asynchronous circuit synthesis [32], we performed this step manually to obtain highly-efficient circuit implementations for the TrueNorth chip.

The logic simulation of our hybrid design was carried out by a combination of a commercial Verilog simulator, Synopsys verilog compiler simulator (VCS), for synchronous circuits and a custom digital simulator, production rule simulator (PRSIM) [33], for asynchronous circuits. PRSIM simulates production rules and communicates with VCS using the Verilog procedural interface (VPI) [34]. When signal values change across the asynchronous–synchronous circuit boundaries, PRSIM and VCS notify each other via the VPI interface.

The input to PRSIM is a netlist based on production rules. A set of production rules can be viewed as a sequence of events/firings. All events are stored in a queue, and when the preconditions of an event become true, a timestamp is attached to that event. If the timestamp of an event coincides with PRSIM's running clock, the event (production rule) is executed. The timestamps of events can be deterministic or can follow a probability distribution. The probability distributions may be random or long-tailed (i.e., most events are scheduled in the near future, while some events are far in the

future). Such flexibility allows PRSIM to simulate the behavior of synchronous circuits, as well as of asynchronous circuits at random or deterministic timing. Whenever an event is executed, PRSIM performs multiple tests to verify correct circuit behavior. These tests include the following.

1) Verify that all events are noninterfering. An event is noninterfering if it does not create a short circuit under any conditions in the context of the design.
2) Verify proper codification on synchronous buses and asynchronous channels.
3) Verify the correctness of expected values of a channel or a bus (optional).
4) Verify that events are stable. In an asynchronous context, an event is considered stable when all receivers acknowledge each signal transition before the signal changes its value again. PRSIM is also capable of evaluating energy, power, and the transient effects of temperature and supply voltage on gate delays.

Since the TrueNorth chip is a very large design, simulating the logic of the entire chip is impractical (due to time and memory limitations). Instead, we dynamically load and simulate (using VCS and PRSIM) only the neurosynaptic cores that are actively used for a given regression. Overall, we ran millions of regression tests to verify the logical correctness of our design.

The physical design of the TrueNorth chip was carried out by a combination of commercial, academic and in-house tools, depending on the target circuit. On one hand, the blocks using conventional synchronous circuits were synthesized, placed, routed, and statically-timed with commercial design tools (Cadence Encounter). On the other hand, the asynchronous circuits were implemented exercising a number of in-house and academic tools. The high-level logic description of asynchronous circuits in the CHP language was manually decomposed into production rules. We then used an academic tool to convert production rules into a parameterized transistor netlist. These transistor-level implementations of the asynchronous circuits were constructed based on a custom cell library built in-house. With the help of in-house tools and Cadence Virtuoso, the mask design engineers created the physical design (layout) of the asynchronous implementations. Each step of converting the asynchronous logic design from CHP to silicon was verified by equivalence and design rule checkers.

QDI asynchronous circuits are delay-insensitive, with an exception of isochronic forks [21], which are verified by PRSIM. Thus, these circuits do not require conventional static timing analysis (STA). However, a signal integrity issue, such as a glitch may result in a deadlock of an asynchronous design. As a result, we took several precautions to improve signal integrity and eliminate glitches. In particular, we minimized the wire cross talk noise, reduced gate-level charge sharing, maximized slew rates and confirmed full-swing transitions on all asynchronous gates. In order to avoid glitching, we enforced several rules throughout the asynchronous design. First, the signal slew rates of asynchronous gates must be faster than 5 V/ns. Second, noise from charge sharing on any output node, plus cross talk noise from neighboring wires must not exceed 200 mV (with the logic nominally running at 1 V supply) to protect succeeding gates from switching prematurely [35]. In order to verify these conditions, we simulated the physical design of all asynchronous circuits using the Synopsys HSIM and Hspice simulators. When these conditions were violated, we exercised various techniques to mitigate the issues, including precharging, buffer insertion, and wire shielding.

Signal integrity in asynchronous circuits is one of the crucial issues to be addressed during design verification. It requires an iterative process alternating between simulation and circuit-level updates, similar to the timing closure process for synchronous circuits. However, signal integrity verification is generally more time-consuming, because electrical circuit simulation takes longer than STA. The necessity for signal integrity analysis in the context of a large asynchronous design (over 5 billion total transistors on the TrueNorth chip) motivates a co-simulation tool that performs electrical simulation of the targeted asynchronous blocks (both pre- and post-physical design), while the remainder of the design is simulated logically, at a faster speed.

We utilized a custom tool, Verilog-HSIM-PRSIM cosimulation environment (COSIM), which enables co-simulation of an arbitrary mix of synchronous and asynchronous circuit families at various levels of abstraction. COSIM automatically creates an interface between the Verilog simulator VCS, the PRSIM asynchronous simulator, and the HSIM transistor-level simulator using VPI. The tool supports simultaneous co-simulation of high-level behavioral descriptions (Verilog, very high speed integrated circuit hardware description language, CHP), RTL netlists and transistor-level netlists using digital simulators (VCS, PRSIM) and an analog transistor-level simulator (HSIM). COSIM contains preloaded communication primitives, including Boolean signals, buses, and asynchronous channels. Whenever a connection needs to be made between different levels of abstraction, the tool detects the type of the interface required, and automatically generates the Verilog code that performs the necessary connection. Furthermore, COSIM has an extensive module library to send, receive, probe and check correctness of communication channels.

B. Timing Design

As discussed earlier, the TrueNorth chip contains several synchronous blocks, including the core computation circuits, memory controller, and scan chain logic. We followed a standard synthesis approach for each individual synchronous block (Fig. 13), however, this presented two unique challenges for system integration. First, there are no automated CAD tools that support mixed asynchronous–synchronous designs, and in particular the interface timing requirements between blocks. And second, we could not use hierarchical STA for timing paths between blocks at the core level, due to the presence of asynchronous blocks in the design.

In order to overcome these challenges, we defined the following interface design methodology.

1) *Synchronous to Synchronous Blocks:* We divided the timing period between the two blocks and interconnecting wire, and assigned timing constraints to each of the blocks for standard STA.

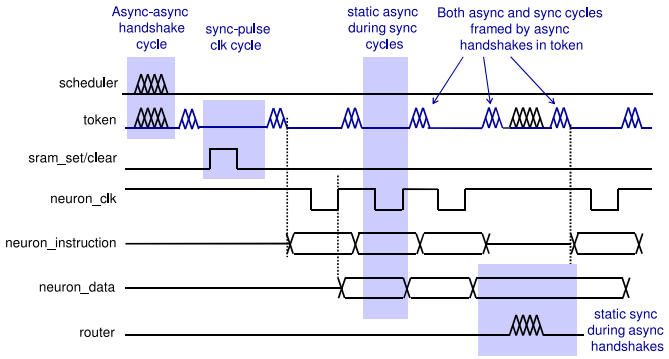

Fig. 14. Mutually exclusive asynchronous/synchronous timing operations.

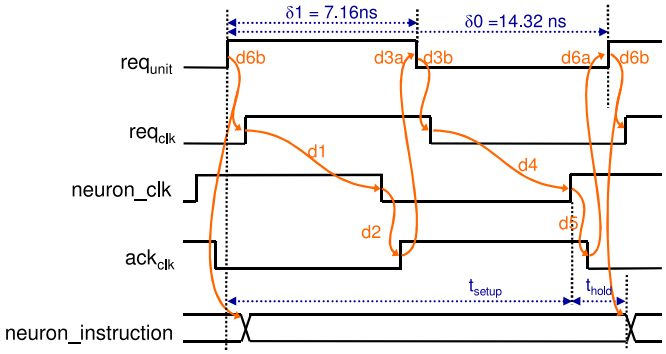

Fig. 15. Token controller/neuron detailed timing diagram: $d1$ and $d4$ are set by the first programmable delay line; $d2$ and $d5$ are set by the second programmable delay line; and $d3$ and $d6$ are the duration of an asynchronous handshake and an OR tree propagation delay ($\sim$1 ns). $t_{\text{setup}} = (d6b + d1 + d2 + d3 + d4)$ and $t_{\text{hold}} = (d5 + d6a)$.

2) *Asynchronous to Asynchronous Blocks:* These interfaces use native QDI asynchronous handshaking and do not require additional timing constraints.
3) *Synchronous to Asynchronous Blocks:* Data is set by the synchronous block one clock cycle prior to consumption by the asynchronous block. Consumption is triggered by the asynchronous token controller following completion of the synchronous clock cycle.
4) *Asynchronous to Synchronous Blocks:* Data is set by the token controller and then the token controller sends a clock pulse (controlled by programmable delays) to latch the data. This interface has a delay assumption described in Fig. 15 in more detail.

Fig. 14 shows the mutually exclusive timing of the mixed synchronous/asynchronous timing domains. Synchronous and asynchronous transactions occur on different cycles, as dictated by the token controller, and both types of cycles are bounded by an asynchronous handshake in the token controller. So for any two interfacing blocks, the token controller assures that the asynchronous signals are static during a cycle while the synchronous signals transition, and the synchronous signals are static during a cycle while the asynchronous signals transition.

The token controller generates the synchronous clock and data according to the timing diagram of Fig. 15. In order to mitigate the risk of a timing violation, we use two programmable delay lines (based on delay cells in the standard cell library) to configure the position of the rising clock edge in increments of 0.27 ns, as shown in Fig. 10. This governs the setup and hold times of the data going into the neuron block.

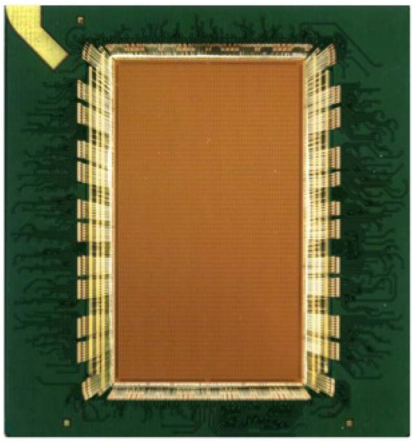

Fig. 16. TrueNorth chip die photo with package substrate.

In general, the interfaces between the asynchronous–synchronous boundaries require special care to assure the correct circuit behavior. Without automated tools to handle this, it is important to limit the asynchronous–synchronous interfaces to the block and unit boundaries to make the design and verification manageable.

C. Manufacturing Considerations

The TrueNorth chip, shown in Fig. 16, was fabricated using Samsung's 28 nm LPP CMOS process technology. This silicon fabrication process was tuned for low-power devices. In order to further reduce power consumption of the chip, we used field-effect transistors with longer channel lengths. In our design, for all asynchronous state-holding gates, we used combinational feedback [33] instead of staticizers [36] to minimize switching power consumption, as well as due to process technology limitation of only few discrete transistor gate lengths.

At 4.3 cm^2 in die area and 5.4 billion transistors, the TrueNorth chip is larger than a typical ASIC chip, and manufacturing defects could have been a serious issue. To minimize the risk of defects at the design mask level, we utilized high-yield manufacturing rules during the physical design. We also implemented various redundancy and testing structures on the TrueNorth chip. For example, at the circuit level, the core SRAM has ten redundant columns and 16 redundant rows to improve the yield. We also placed dummy cells around the SRAM cell arrays to improve the lithographic rendering quality. Finally, we applied Monte Carlo simulation to the SRAM circuits to verify sufficiently high yield.

Testing and debugging were some of the reasons why we designed the TrueNorth chip to be one-to-one equivalent with its software simulator. In the event of failure, we can compare the chip's behavior to that of the simulator, quickly isolating the deviation and identify the faulty cores. This is one advantage of using digital circuits over analog circuits in designing artificial neurons.

The arrangement of our communication network addresses the yield issues that may arise during fabrication. Faulty cores can be disabled and routed around, allowing the majority of the chip to remain functional in the face of defects. In the less likely case of a faulty router, the entire row and column of cores associated with a given router would have to be disabled, since spikes traveling to any core on the given row/column may pass throughout the defective router and potentially deadlock the system. In these cases, the configuration of the TrueNorth chip must be adjusted by the chip's software to avoid the faulty cores. Therefore, due to the high level of on-chip core redundancy, the chip still remains functional, losing only a fraction of the available cores.

The takeaway point is that while designing the TrueNorth chip we used a number of architectural, circuit-level, as well as manufacturing process-related techniques to make the chip power-efficient and resilient to manufacturing defects.

VII. Measured TrueNorth Chip Data

We tested the TrueNorth chip for logical correctness and extensively characterized it for performance and power consumption [1], [3]. Naturally, the performance and efficiency of the TrueNorth chip varies with the neural activity, routing network connectivity, and operating voltage. Here we present some important technical characteristics of the TrueNorth chip. As explained in Section IV, based on the TrueNorth chip's event-driven implementation, we are able to operate the chip at lower voltages without running into timing violations that one would encounter using a common synchronous design approach. Overall, the TrueNorth chip is operational from 1.05 V down to 0.7 V, with total power consumption ranging from 42 mW in the low corner (0.70 V, 0 Hz firing rate, 0 synapses/neuron) to 323 mW in the high corner (1.05 V, 200 Hz firing rate, 256 synapses/neuron). As an example, to show the chip's low-power capability, we pick an operating point of 0.75 V. At this supply voltage, the maximum computational speed of the chip is 58 GSOPS and the maximum computational energy efficiency is 400 GSOPS/W.[2] While running a typical complex recurrent neural network at 0.75 V with 20 Hz average firing rate and 128 active synapses per neuron at real-time (1 kHz tick), the TrueNorth chip consumes only 65 mW and delivers 46GSOPS/W. We also demonstrated the TrueNorth chip properly functioning faster than real-time (tick $>$ 1 kHz). In our experiments we measured up to 21$\times$ real-time operation, dependent on the activity rates, synaptic density, and voltage levels.

Fig. 17 shows a breakdown of the components of the overall TrueNorth chip power (@ 0.8 V) for three complex recurrent networks with 128 synapses per neuron average, and three different average firing rates. Such complex networks approximate a worst-case scenario for power consumption as the cores are randomly placed on the chip. Random placement significantly increases internal communication power as compared with the application examples where communicating cores tend to be near each other (the placement can be optimized as in Section X). Computation and communication are both event driven, thus active power scales with firing rate. In [3], we measured two to three orders of magnitude speedup in execution time and five orders of magnitude lower energy consumption for a set of computer vision neural networks run on TrueNorth, over the same networks run on a dual 2.4 GHz E5-2440 processor x86 system, as well as a Blue Gene/Q system with up to 32 compute cards.

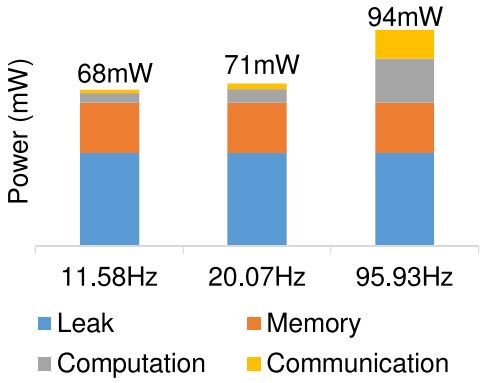

Fig. 17. Total TrueNorth chip power breakdown (@ 0.8 V) for three complex recurrent networks with 128 synapses per neuron average, and three different average firing rates.

From the above data, one can see that the TrueNorth chip is a very power-efficient (65 mW typical) and highly-parallel neurosynaptic chip that runs complex neural networks in real-time. Next, we present the TrueNorth-based systems that we used for collecting the test data, as well as for running the visual recognition applications described in Section IX.

VIII. TrueNorth Platforms

We architected the TrueNorth chip with scalability as one of the major requirements. Like the cores within a chip, the chips themselves are designed to be tiled into a scalable 2-D array without any modification to the underlying routing algorithm or the need for off-chip interface circuits. This way we can create large networks of neurosynaptic cores with many interconnected neurons. From a logical point of view, there is no difference whether the communication between neurons occurs on the same chip or across many chips.

Using these principles, we built a single-chip testing system for characterizing the TrueNorth chips, as well as several multichip systems, with various arrangements of the TrueNorth chips [3]. We also built a mobile single TrueNorth chip system (NS1e), which is designed as a stand-alone, compact, low-power, flexible delivery platform for real-time spiking neural network applications. The multichip systems include a 4×1 chip platform and a 4×4 chip platform [3]. The total system power, while running the grid classifier application on the 4×4 platform at real time was 7.2 W, of which the TrueNorth array (@ 1.0 V) consumed 2.5 W. These TrueNorth systems are currently being successfully used for the neural applications described in Section IX.

The direct communication between chips in all directions on multichip boards occurs by means of asynchronous

[2]Measured using a recurrent network with neuron firing rates in the range of 0–200 Hz and synaptic connectivity varying between 0–100%.

bundled-data protocol, as explained in Section V-G. The TrueNorth pads are allocated and aligned in a manner that allows simple tiling and direct chip-to-chip signal connection. The communication interface signals from the peripheral chips are routed to off-board connectors to allow further scaling that creates a larger 2-D mesh of neurosynaptic cores by tiling (or densely stacking) the TrueNorth boards. These off-board connectors may be mated directly to similar connectors on other boards for further seamless chip communication or through FPGA-based data-port expanders to create an even larger scale routing infrastructure.

The key for building more complex TrueNorth systems in the future, as proposed in Section XI, is the TrueNorth chip's power efficiency, which relaxes the system power delivery constraints and significantly decreases (or potentially eliminates) the cooling requirements.

IX. TrueNorth Chip Applications

Applications for the TrueNorth architecture are developed using the CPE, an object-oriented, compositional language and development environment for building neurosynaptic software that is efficient, modular, scalable, reusable, and collaborative [4]. We previously demonstrated a variety of corelet applications, including speaker recognition, musical composer recognition, digit recognition, hidden Markov model sequence modeling, collision avoidance, and eye detection [5].

Corelet programs describe neurosynaptic cores in a logical format that is agnostic to the physical location of each core in the actual hardware, allowing the same corelet to be deployed on different hardware configurations without modifying the source code. However, for any given hardware configuration, every logical core must be mapped to a unique physical core on a TrueNorth chip. These physical core placements can be optimized to minimize bandwidth and active power using application-agnostic algorithms, as described in Section X.

To demonstrate these optimizations, we present a selection of streaming video applications (Fig. 18), all of which can process video at 30 frames/s in real-time [3].

1) *Haar-Like Features:* Haar-like features, popular for face recognition [37], are differences of summed intensity in adjacent rectangles. We extract ten Haar-like feature maps—eight sensitive to vertical or horizontal edges with various phases, plus two center-surround features—from each 200×100-pixel frame.
2) *Local Binary Patterns:* Local binary patterns (LBPs) are simple texture features used in biometrics, robot navigation, and magnetic resonance imaging analysis [38]. We extract 20-bin LBP feature histograms from subpatches covering a 200×100-pixel frame.
3) *Saccade Generator:* Mammalian visual attention directs a succession of quick eye movements called saccades to salient targets in the field of view [39]. Our application identifies saccade targets in an HD image by combining various feature maps into a single saliency map, performing a winner-take-all operation to find the most salient pixels, and using inhibition of return to prevent consecutive saccades from looking at the same place.
4) *K-Means Classifier:* Once a saccade generator has successfully detected and centered a salient object in a 32×32-pixel patch, our first classifier example uses K-means clustering to identify up to one foreground object per frame [40].
5) *Grid Classifier:* To detect and classify all objects in an entire image simultaneously, our second two classifier examples tile the image with a grid of identically trained patch classifiers. Grid classifier B [Fig. 18(g)] is tuned to maximize classification accuracy, using 16 chips to obtain an out-of-sample precision of 0.85 and 0.75 recall on the DARPA Neovision2 Tower dataset, a multiobject detection and classification task based on a collection of fixed-camera HD videos containing moving and stationary people, bicyclists, cars, buses, and trucks [41]. Grid Classifier A trades a small amount of accuracy for much lower chip area and energy consumption, requiring only 7 chips to obtain a 0.82 precision and 0.71 recall.

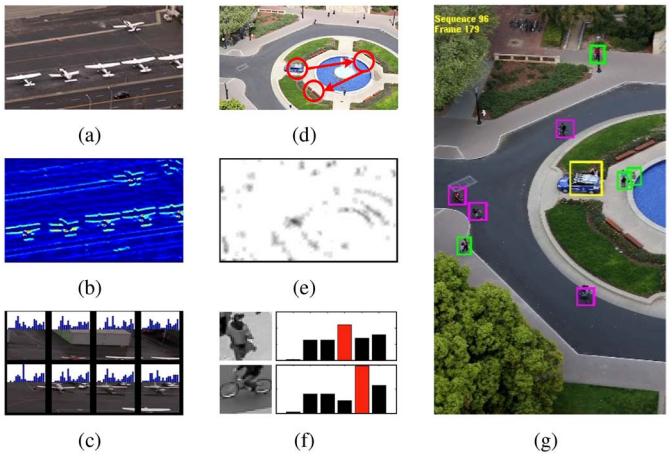

Fig. 18. Examples of streaming video applications. Left: features extracted from (a) one frame of a moving-camera video. (b) Haar-feature response map for horizontal lines. (c) Eight LBP histograms extracted from eight subpatches. Middle: (d) consecutive saccades, targeted to peaks of a (e) saliency map. (f) K-means classifier predictions for a person (top, fourth bar) or bicyclist (bottom, fifth bar); bar height denotes prediction strength for five foreground classes and a null class (sixth bar). Right: (g) grid classifier detects and classifies five types of foreground objects in each frame of an HD video with a vertical aspect ratio (green box = person, magenta = bicyclist, and yellow = car).

The total TrueNorth power while running each of these applications at real-time is listed in the final column of Table I. While the primary application that we present is our solution to the NeoVision visual-object-recognition task, which contains what/where pathways analogous to the mammalian visual cortex, TrueNorth is a general-purpose computing architecture whose design space is not confined strictly to biologically motivated algorithms. Thus, while the bio-inspired NeoVision application demonstrates depth, applications like Haar features, LBP histograms, and K-means are intended to show breadth across a range of designs. For a case study on how conventional computer-vision algorithms like Haar features can efficiently map to the TrueNorth architecture in a saliency-map application (see [42]).

TABLE I
OPTIMIZATION RESULTS AND MEASURED TOTAL TRUENORTH CHIP POWER RESULTS

	Network Size				Communication Power (Active)			Total Power
Applications	#chips	#cores	#nets	#pins	Default	CPLACE	× Improvement	(Watts)
Saccade generator	1	2297	17523	35046	0.40mW	0.11mW	3.6×	0.049W @0.75V
Local Binary Patterns	1	3520	22032	44064	4.41mW	2.33mW	1.9×	0.064W @0.75V
Haar-like features	1	3875	45822	91644	7.57mW	1.81mW	4.2×	0.058W @0.75V
K-means classifier	4	14832	445380	890760	20.03mW	6.44mW	3.1×	0.653W @1.0V
Grid classifier A	7	27644	461788	923576	589.90mW	130.88mW	4.5×	1.274W @1.0V
Grid classifier B	16	62375	960013	1920030	353.95mW	70.06mW	5.1×	2.515W @1.0V

X. ALGORITHM MAPPING: CORE PLACEMENT

The power consumption of a TrueNorth system running a program depends on how the program is mapped on the chip. The energy required to communicate a spike between cores or even between chips increases with the distance between the source and the destination. Mapping logically connected cores to physically adjacent cores on the same chip can dramatically reduce the power consumption of the overall system. We adopted an existing VLSI design automation tool to solve this problem.

The problem of mapping neurosynaptic cores to minimize spike communication power can be formulated as a wire-length minimization problem in VLSI placement [43]

$$\begin{aligned} &\min\ W(\mathbf{x},\mathbf{y}) \\ &\text{s.t. } D_i(\mathbf{x},\mathbf{y}) \le M_i, \qquad \text{for each tile } i \end{aligned}$$

where $W(\mathbf{x},\mathbf{y})$ is a wire-length function and $D_i(\mathbf{x},\mathbf{y})$ and M_i encode placement legality constraints for floor-planning tile i. For neurosynaptic core placement, $D_i(\mathbf{x},\mathbf{y})$ is a potential function representing the sum of core areas in tile i, and M_t is the maximum potential value (i.e., the maximum number of cores) allowed in a tile.

To define a wire-length function for neurosynaptic core placement, we represent the network as a graph $G = (V, E)$ whose nodes $V = \{v_1, v_2, \ldots, v_n\}$ denote cores, and edges $E = \{e_1, e_2, \ldots, e_m\}$ denote nets connecting cores. For nets between cores on the same chip, the active power per spike is a linear function of the Manhattan distance between the source and the destination. Nets connecting cores on different chips consume substantially higher power while bridging chip boundaries. Accordingly, our net wire-length model has separate terms for within-chip (n_{on}) and off-chip (n_{off}) nets

$$W(\mathbf{x},\mathbf{y}) = \sum_{e\in n_{\text{on}}} \left[|x_i - x_j| + |y_i - y_j|\right] + \sum_{e\in n_{\text{off}}} \mathbf{E} * \left[|cx_i - cx_j| + |cy_i - cy_j|\right]$$

where x_i and y_i are the respective x- and y-coordinates of the core v_i, $\mathbf{E}$ is a large constant penalty for off-chip communication between neighboring chips, and cx_i and cy_i denote the coordinates of chip boundaries. The off-chip term is simply the penalty constant multiplied by the number of chip boundaries a spike must traverse.

Recall from Section V-G, each merge operation combines 32 buses down to one bus, followed by a two times serialization, resulting in a 64 times reduction in boundary bandwidth. In addition, there is a greater than ten times disparity in frequency between the internal NoC frequency and the I/O pad frequency. Combined, the spike bandwidth is over 640 times lower at the chip boundary than internal to the chip.

In addition to its static wire-length $W(\mathbf{x},\mathbf{y})$, each edge e_i is assigned a weight w_i that is proportional to the number of spikes communicated across that edge. Minimizing total weighted wire-length for a network of neurosynaptic cores is equivalent to minimizing active communication power of a TrueNorth system.

To perform the minimization, we use IBM CPLACE [44], an analytical placer used to design high performance server microprocessors, gaming chips, and other ASICs. When placing each network, we use four different global placement algorithms and use the result with the minimum wire length as the actual implementation.[3] In practice, all four methods produce efficient placement results.

1) *Multilevel Partitioning-Driven Algorithm [46]:* This method recursively partitions the placement area into subregions and minimizes the crossing nets called "cuts" among the subregions. This process is repeated for each subregion until the area becomes small enough.
2) *Analytical Constraint Generation Algorithm [47]:* The partitioning-based algorithm is enhanced with cell-distribution constraints determined by an analytical free space calculation.
3) *Hierarchical Quadratic Placement Algorithm [48]:* The analytical top-down quadratic partitioning algorithm by Vygen [49] is enhanced with a semi-persistent bottom-up clustering algorithm.
4) *Quadric-Based Force-Directed Analytical Algorithm [44]:* The force-directed analytical placer is one of the most popular placement algorithms, balancing wire-length minimization with density constraints to yield a high quality solution.

Table I lists placement optimization results and measured total power for various applications mapped to either the single-chip TrueNorth testing board or the 4×4 TrueNorth board, corresponding to networks on the order of 0.6–16 million neurons and 150 million to 4 billion synapses. For grid classifier *B*, a 16-chip network mapped to the 4×4 board, optimizing core placement reduces average static hop distance by a factor of 24× to only 6.7 hops (Fig. 19), and reduces

[3]Note that the physical mapping problems are easily ported into the placement bookshelf contest formats [45]. Therefore, if you are interested in trying your own placement implementation, please contact the authors.

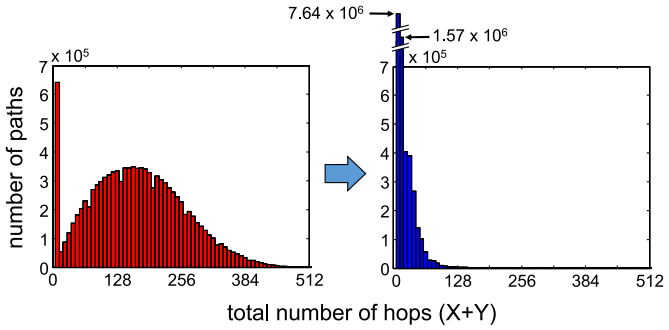

Fig. 19. Hop distribution of spikes using unoptimized (left) or optimized (right) core placement of grid classifier *B*.

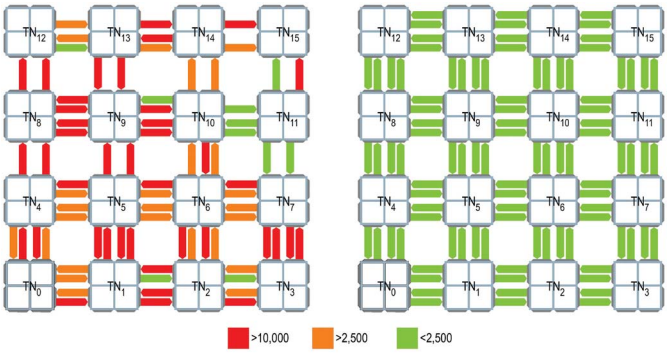

Fig. 20. Maximum spikes per tick for each port using unoptimized (left) or optimized (right) core placement of grid classifier *B* on the 4×4 TrueNorth board.

the number of static cross-chip routes by a factor of 55×. Optimization reduces maximum spike traffic from about 10K spikes/tick on some ports down to 2500 spikes/tick on all ports (Fig. 20). This optimization first ensures the boundary traffic is low enough to avoid congestion delay, while maintaining real-time operation. Further optimization reduces the energy consumption.

The minimum wire length solution also implicitly addresses the worst-case spike travel time by reducing congestion, as well as by reducing the number of long wire paths, both within and across chip boundaries, as shown in the hop distance distributions in Fig. 19.

This optimization technique of mapping logical cores to physical TrueNorth cores is invaluable for the software design methodology, improving the efficiency of TrueNorth programs run on hardware. It both reduces network bandwidth requirements and minimizes active communication power.

XI. Future Large Scale TrueNorth Systems

This section discusses our perspective for scaling the TrueNorth architecture to massive neurosynaptic systems using the existing TrueNorth hardware described earlier. The extremely low-power profile of the TrueNorth chip enables us to envision large-scale neurosynaptic systems approaching the scale of a mammalian brain.

Many neural networks employ both short and long connections for communication between neurons. On a TrueNorth-based large-scale system, short distance communication is handled by the core crossbars and asynchronous routers at the intrachip level. For longer distance communication, which may connect neurons that are physically located far apart from each other in the system, we use robust asynchronous channels to communicate between chips. As for the ultralong distance connections, we use conventional communication protocols. As one design solution, we can create a tree-like topology of Ethernet nodes. On each node, programmable-logic is used to convert asynchronous spike information from the periphery of a 2-D mesh network of TrueNorth chips to a TCP/IP packet format, which can be sent over the Ethernet.

With low-power consumption being the major characteristic of our neurosynaptic chips, TrueNorth chips may be densely packed together without cooling or power distribution concerns. Our preliminary calculations show that 4096 chips may be packed into a single rack, creating a 4 billion neuron system. The estimated power consumption for the TrueNorth chips on such a system is 300 W. For communication to the outside world, as mentioned earlier, and for fast configuration purposes, TrueNorth systems need to employ FPGAs and network interfaces. As a result of this power consumption overhead, we estimate the total power of a 4 billion neuron system at a few kilowatts. Scaling such a TrueNorth system further to 96 racks will deliver 412 billion neurons with 10^{14} synapses, which is comparable to a human brain in terms of the number of neurons and synapses. At this scale, the power consumption attributed to the TrueNorth chips is estimated to be 29 kW, although the total system power is likely to be a few hundred kilowatts due to communication and configuration devices. Such a system should be able to compute in real-time the 10^{14} synapse model, which was previously simulated by the BlueGene/Q Sequoia 7.9 MW system 1542× slower than real-time [10].

XII. Conclusion

In this paper, we presented the design and tool flow innovations of the groundbreaking TrueNorth chip that implements our brain-inspired, event-driven, non-von Neumann architecture. The TrueNorth chip is the world's first 1 million neuron, 256 million synapse fully-digital neurosynaptic chip, implemented in a standard-CMOS manufacturing process. The chip is highly-parallel, defect-tolerant, and operates at real-time with an extremely low typical power consumption of 65 mW. Inside our low-power TrueNorth chip, we implement architectural innovations and complex unconventional event-driven circuit primitives to adhere to our seven main design principles.

We covered the mixed asynchronous–synchronous design approach, and the methodology that we developed while creating the TrueNorth chip. This novel tool flow, along with the event-driven techniques that we demonstrated, give designers an opportunity to mix asynchronous and synchronous circuits in order to make their future state-of-the-art designs more flexible and energy efficient. We now turn to the CAD research community for collaboration in automating and improving this tool flow further.

In contrast to general purpose von Neumann machines, which are optimized for high-precision integer and floating-point operations, TrueNorth broadly targets sensory information processing, machine learning, and cognitive computing applications using synaptic operations. However, like the early von Neumann machines, the task is now to create efficient neurosynaptic systems and optimize them in terms of programming models, algorithms, and architectural features. Using the TrueNorth chip, we can create large-scale and low-power cognitive systems as a result of the architecture's scalability property. We have already built the first multichip TrueNorth-based neurosynaptic platforms (with up to 16 million neurons and 4 billion synapses) and demonstrated example applications, such as visual object recognition, running on these platforms at real-time with orders of magnitude lower power consumption than conventional processors. With such outstanding technical characteristics, TrueNorth systems significantly advance the field of cognitive research and revolutionize the current state of real-world multisensory systems, creating new opportunities for applications ranging from robotics to battery-powered consumer electronics to large-scale analytics platforms.

ACKNOWLEDGMENT

The authors would like to thank several collaborators: A. Agrawal, C. Alpert, A. Amir, A. Andreopoulos, R. Appuswamy, S. Asaad, C. Baks, D. Barch, R. Bellofatto, D. Berg, H. Baier, T. Christensen, C. Cox, S. Esser, M. Flickner, D. Friedman, S. Gilson, C. Guo, S. Hall, R. Haring, C. Haymes, J. Ho, T. Horvath, K. Inoue, S. Iyer, J. Kusnitz, S. Lekuch, Z. Li, J. Liu, M. Mastro, E. McQuinn, S. Millman, R. Mousalli, D. Nguyen, H. Nguyen, T. Nguyen, N. Pass, K. Prasad, C. Ortega-Otero, Z. Saliba, K. Schleupen, J.-s. Seo, B. Shaw, K. Shimohashi, A. Shokoubakhsh, F. Tsai, J. Tierno, K. Wecker, S.-y. Wang, I. Vo, and T. Wong. The views, opinions, and/or findings contained in this paper/presentation are those of the author(s)/presenter(s) and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

REFERENCES

[1] P. A. Merolla *et al.*, "A million spiking-neuron integrated circuit with a scalable communication network and interface," *Science*, vol. 345, no. 6197, pp. 668–673, Aug. 2014.

[2] Y. Bengio, "Learning deep architectures for AI," *Found. Trends Mach. Learn.*, vol. 2, no. 1, pp. 1–127, 2009.

[3] A. S. Cassidy *et al.*, "Real-time scalable cortical computing at 46 giga-synaptic OPS/watt with $\sim$ 100$\times$ speedup in time-to-solution and $\sim 100,000\times$ reduction in energy-to-solution," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal. (SC)*, New Orleans, LA, USA, 2014, pp. 27–38.

[4] A. Amir *et al.*, "Cognitive computing programming paradigm: A Corelet language for composing networks of neuro-synaptic cores," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, Dallas, TX, USA, 2013, pp. 1–10.

[5] S. K. Esser *et al.*, "Cognitive computing systems: Algorithms and applications for networks of neurosynaptic cores," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, Dallas, TX, USA, 2013, pp. 1–10.

[6] D. S. Modha and R. Singh, "Network architecture of the long distance pathways in the Macaque brain," *Proc. Nat. Acad. Sci. USA*, vol. 107, no. 30, pp. 13485–13490, 2010.

[7] R. Ananthanarayanan and D. S. Modha, "Anatomy of a cortical simulator," in *Proc. Supercomput.*, Reno, NV, USA, 2007, pp. 1–12.

[8] R. Ananthanarayanan, S. K. Esser, H. D. Simon, and D. S. Modha, "The cat is out of the bag: Cortical simulations with 10^9 neurons, 10^{13} synapses," in *Proc. Conf. High Perform. Comput. Netw. Storage Anal. (SC)*, Portland, OR, USA, 2009, pp. 1–12.

[9] R. Preissl *et al.*, "Compass: A scalable simulator for an architecture for cognitive computing," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, Salt Lake City, UT, USA, 2012, pp. 1–11.

[10] T. M. Wong *et al.*, "10^{14}," IBM Res. Div., Almaden Res. Center, San Jose, CA, USA, Tech. Rep. RJ10502, 2012.

[11] J. Hsu, "How IBM got brainlike efficiency from the TrueNorth chip," *IEEE Spectr.*, vol. 51, no. 10, pp. 17–19, Sep. 2014. [Online]. Available: http://spectrum.ieee.org/computing/hardware/how-ibm-got-brainlike-efficiency-from-the-truenorth-chip

[12] E. Painkras *et al.*, "SpiNNaker: A 1-W 18-core system-on-chip for massively-parallel neural network simulation," *IEEE J. Solid-State Circuits*, vol. 48, no. 8, pp. 1943–1953, Aug. 2013.

[13] E. Stromatias, F. Galluppi, C. Patterson, and S. Furber, "Power analysis of large-scale, real-time neural networks on SpiNNaker," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, Dallas, TX, USA, 2013, pp. 1–8.

[14] B. V. Benjamin *et al.*, "Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations," *Proc. IEEE*, vol. 102, no. 5, pp. 699–716, May 2014.

[15] P. Merolla, J. Arthur, R. Alvarez, J.-M. Bussat, and K. Boahen, "A multicast tree router for multichip neuromorphic systems," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 61, no. 3, pp. 820–833, Mar. 2014.

[16] J. Schemmel *et al.*, "Live demonstration: A scaled-down version of the BrainScaleS wafer-scale neuromorphic system," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Seoul, Korea, 2012, p. 702.

[17] J. Park, S. Ha, T. Yu, E. Neftci, and G. Cauwenberghs, "A 65k-neuron 73-Mevents/s 22-pJ/event asynchronous micro-pipelined integrate-and-fire array transceiver," in *Proc. IEEE Biomed. Circuits Syst. Conf. (BioCAS)*, Lausanne, Switzerland, 2014, pp. 675–678.

[18] S. Moradi and G. Indiveri, "An event-based neural network architecture with an asynchronous programmable synaptic memory," *IEEE Trans. Biomed. Circuits Syst.*, vol. 8, no. 1, pp. 98–107, Mar. 2013.

[19] A. S. Cassidy, J. Georgiou, and A. G. Andreou, "Design of silicon brains in the nano-CMOS era: Spiking neurons, learning synapses and neural architecture optimization," *Neural Netw.*, vol. 45, pp. 4–26, Sep. 2013.

[20] A. S. Cassidy *et al.*, "Cognitive computing building block: A versatile and efficient digital neuron model for neurosynaptic cores," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, Dallas, TX, USA, 2013, pp. 1–10.

[21] A. J. Martin, "The limitations to delay-insensitivity in asynchronous circuits," in *Proc. ARVLSI*, Sydney, NSW, Australia, 1990, pp. 263–278.

[22] C. G. Wong and A. J. Martin, "High-level synthesis of asynchronous systems by data-driven decomposition," in *Proc. Design Autom. Conf.*, Anaheim, CA, USA, 2003, pp. 508–513.

[23] D. E. Muller and W. S. Bartky, "A theory of asynchronous circuits," in *Proc. Int. Symp. Theory Switch.*, Cambridge, MA, USA, 1959, pp. 204–243.

[24] A. M. Lines, "Pipelined asynchronous circuits," Dept. Comput. Sci., California Inst. Technol., Pasadena, CA, USA, Tech. Rep. CaltechCSTR:1998.cs-tr-95-21, 1998.

[25] E. M. Izhikevich, "Which model to use for cortical spiking neurons?" *IEEE Trans. Neural Netw.*, vol. 15, no. 5, pp. 1063–1070, Sep. 2004.

[26] Y. Shin *et al.*, "28nm high-metal-gate heterogeneous quad-core CPUs for high-performance and energy-efficient mobile application processor," in *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, San Francisco, CA, USA, 2013, pp. 154–155.

[27] J. V. Arthur *et al.*, "Building block of a programmable neuromorphic substrate: A digital neurosynaptic core," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Brisbane QLD, Australia, 2012, pp. 1–8.

[28] D. Fang, S. Peng, C. LaFrieda, and R. Manohar, "A three-tier asynchronous FPGA," in *Proc. Int. VLSI/ULSI Multilevel Interconnect. Conf.*, Fremont, CA, USA, 2006, pp. 1–8.

[29] A. J. Martin, "Compiling communicating processes into delay-insensitive VLSI circuits," *Distrib. Comput.*, vol. 1, no. 4, pp. 226–234, Dec. 1986.

[30] R. Manohar, "An analysis of reshuffled handshaking expansions," in *Proc. 7th Int. Symp. ASYNC*, Salt Lake City, UT, USA, Mar. 2001, pp. 96–105.

[31] A. J. Martin and M. Nystrom, "CAST: Caltech asynchronous synthesis tools," in *Proc. 4th Asynchronous Circuit Design Work. Group Workshop*, Turku, Finland, Jun. 2004.

[32] S. M. Burns and A. J. Martin, "Syntax-directed translation of concurrent programs into self-timed circuits," Dept. Comput. Sci., California Inst. Technol., Pasadena, CA, USA, Tech. Rep. CaltechCSTR:1988.cs-tr-88-14, 1988.
[33] R. Manohar, "Asynchronous VLSI systems," Class Materials for ECE 574, Cornell Univ., Ithaca, NY, USA, 1999.
[34] *IEEE Standard for Verilog Hardware Description Language*, IEEE Standard 1364-2005, 2006.
[35] K. L. Shepard and V. Narayanan, "Noise in deep submicron digital design," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, San Jose, CA, USA, 1996, pp. 524–531.
[36] M. Nystrom, "Asynchronous pulse logic," Ph.D. dissertation, Dept. Eng. Appl. Sci., California Inst. Technol., Pasadena, CA, USA, 2001.
[37] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. Comput. Vis. Pattern Recognit.*, Kauai, HI, USA, 2001, pp. I-511–I-518.
[38] T. Ojala, M. Pietikainen, and T. Maenpaa, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, Jul. 2002.
[39] A. Andreopoulos and J. K. Tsotsos, "50 years of object recognition: Directions forward," *Comput. Vis. Image Und.*, vol. 117, no. 8, pp. 827–891, Aug. 2013.
[40] A. Coates and A. Y. Ng, "The importance of encoding versus training with sparse coding and vector quantization," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Bellevue, WA, USA, 2011, pp. 921–928.
[41] R. Kasturi *et al.*, "Performance evaluation of neuromorphic-vision object recognition algorithms," in *Proc. 22nd Int. Conf. Pattern Recognit. (ICPR)*, Stockholm, Sweden, Aug. 2014, pp. 2401–2406.
[42] A. Andreopoulos *et al.*, "Visual saliency on networks of neurosynaptic cores," *IBM J. Res. Develop.*, vol. 59, nos. 2–3, pp. 9:1–9:16, Mar./May 2015.
[43] J. Cong and G.-J. Nam, *Modern Circuit Placement: Best Practices and Results*. Boston, MA, USA: Springer, 2007.
[44] N. Viswanathan *et al.*, "RQL: Global placement via relaxed quadratic spreading and linearization," in *Proc. ACM/IEEE Design Autom. Conf.*, San Diego, CA, USA, 2007, pp. 453–458.
[45] G.-J. Nam, C. J. Alpert, P. Villarrubia, B. Winter, and M. Yildiz, "The ISPD2005 placement contest and benchmark suite," in *Proc. Int. Symp. Phys. Design*, San Francisco, CA, USA, 2005, pp. 216–220.
[46] A. E. Caldwell, A. B. Kahng, and I. L. Markov, "Can recursive bisection alone produce routable placements?" in *Proc. ACM/IEEE Design Autom. Conf.*, Los Angeles, CA, USA, 2000, pp. 693–698.
[47] C. J. Alpert, G.-J. Nam, and P. G. Villarrubia, "Effective free space management for cut-based placement via analytical constraint generation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 22, no. 10, pp. 1343–1353, Oct. 2003.
[48] G.-J. Nam, S. Reda, C. J. Alpert, P. G. Villarrubia, and A. B. Kahng, "A fast hierarchical quadratic placement algorithm," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 4, pp. 678–691, Apr. 2006.
[49] J. Vygen, "Algorithms for large-scale flat placement," in *Proc. ACM/IEEE Design Autom. Conf.*, Anaheim, CA, USA, 1997, pp. 746–751.

Filipp Akopyan (M'04) was born in Moscow, Russia. He received the bachelor's degree in electrical engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 2004, and the Ph.D. degree in electrical and computer engineering (with a minor in applied mathematics) from Cornell University, Ithaca, NY, USA, in 2011.

He joined the IBM Brain-Inspired Computing Group led by D. Modha. During his IBM tenure, he was one of the lead engineers who created the world's most advanced neuromorphic chip, TrueNorth, as part of the DARPA SyNAPSE program, and he is a key contributor to the DARPA Cortical Processor program. One of his main contributions was leading the complex task of designing, simulating and verifying mixed synchronous—asynchronous circuits to implement the TrueNorth chip correctly and efficiently. He has also been designing low-power cognitive systems and advancing neural algorithm development for various multisensory applications. He has authored several highly cited publications, with over 400 total citations, in the areas of asynchronous design, signal processing, neuromorphic computing, and low-power chip design.

Dr. Akopyan was a recipient of numerous awards and honors in the fields of electrical and computer engineering.

Jun Sawada (M'01) received the B.S. and M.S. degrees in mathematics from Kyoto University, Kyoto, Japan, and the Ph.D. degree in computer sciences from the University of Texas at Austin, Austin, TX, USA, for the study in formal verification of hardware, VLSI microarchitecture, automated theorem proving and automated reasoning.

He has been a Research Staff Member at IBM Research—Austin, Austin, TX, USA, since 2000. At IBM, he participated in many chip development projects, including the ones for power microprocessors and BlueGene supercomputers, and pioneered the application of formal verification technologies to large-scale microprocessor designs. Recently, he has been researching on the neurosynaptic chip and system designs, and co-lead the development team of the TrueNorth chip.

Andrew Cassidy (M'97) received the M.S. degree in electrical and computer engineering from Carnegie Mellon University, Pittsburgh, PA, USA, in 2003, and the Ph.D. degree in electrical and computer engineering from Johns Hopkins University, Baltimore, MD, USA, in 2010.

He is a member of the Cognitive Computing group at IBM Research—Almaden, San Jose, CA, USA, and is engaged in the DARPA funded SyNAPSE project. His expertise is in non-traditional computer architecture, and he is experienced in architectural optimization, design, and implementation, particularly for machine learning algorithms and cognitive computing applications. He has authored over 25 publications in international journals and conferences.

Rodrigo Alvarez-Icaza received the B.S. degree in mechanical and electrical engineering from Universidad Iberoamericana, and the Ph.D. degree in neuromorphic engineering and biological systems from the University of Pennsylvania, Philadelphia, PA, USA, under the supervision of Dr. K. Boahen, and Stanford University, Stanford, CA, USA.

He co-founded an Internet startup after completing the B.S. degree, and later spent a year of independent research in the fields of robotics and A.I., eventually focusing on nontraditional neural networks. He also researched sensory-motor learning based on activity-dependent growth of neural connections, with the long-term ambition of embedding the computation in application-specific hardware. This work led to a teaching position at Universidad Iberoamericana, where he founded the Robotics Laboratory. He is currently with IBM Research.

John Arthur (M'05) received the B.S.E. degree in electrical engineering from Arizona State University, Tempe, AZ, USA, in 2000, and the Ph.D. degree in bioengineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2006.

He is a Research Scientist in the Brain-Inspired Computing Group at IBM Research—Almaden, San Jose, CA, USA. His current research interests include neuromorphic and neurosynaptic systems, building many of the largest hardware spiking neural systems, such as TrueNorth. He held a post-doctoral position in bioengineering at Stanford University, Stanford, CA, USA, until 2010, where he was involved in the Neurogrid Project, which aims to provide neuroscientists with a desktop supercomputer for modeling large networks of spiking neurons.

Paul Merolla received the B.S. degree in electrical engineering from the University of Virginia, Charlottesville, VA, USA, in 2000, and the Ph.D. degree in bioengineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2006.

He was a post-doctoral scholar with Stanford's Brains in Silicon Laboratory, Stanford University, Stanford, CA, USA, from 2006 to 2010. He is currently a Research Scientist with IBM's Brain-Inspired Computing Group, IBM Research—Almaden, San Jose, CA, USA. His research is focused on building more intelligent computers, drawing inspiration from neuroscience and machine learning. He has been a lead designer on over 10 neuromorphic chips, including Neurogrid (a sixteen-chip system that emulates one-million analog neurons in real time) and more recently, IBM's cognitive computing chips, as part of the DARPA SyNAPSE program. His current research interests include developing algorithms for these chips aimed at solving real world problems.

Nabil Imam received the Ph.D. degree from Cornell University, Ithaca, NY, USA, in 2014.

Since then, he has been a member of the Brain-Inspired Computing Group, IBM Research—Almaden, San Jose, CA, USA. His current research interests include analysis of neural algorithms instantiated within hippocampal and olfactory system microcircuits with the goal of constructing various forms of attractor networks within neuromorphic hardware, brain-inspired approaches to machine learning, in the structural analysis of brain networks, and in the study of neural coding across different spatial and temporal scales.

Yutaka Nakamura received the B.S. degree from the Department of Mechanical Engineering, Waseda University, Tokyo, Japan, in 1985.

He then joined IBM Japan, Tokyo, Japan. In 2012, he joined IBM Research—Tokyo, Tokyo. His current research interests include memory circuit design, redundancy repair, ECC, and neuromorphic chip/system.

Pallab Datta (M'08) received the Ph.D. degree from Iowa State University, Ames, IA, USA, in 2005, under the supervision of Prof. A. K. Somani.

He was with NeuroSciences Institute, La Jolla, CA, USA, and Los Alamos National Laboratory, Los Alamos, NM, USA, and a Visiting Researcher with INRIA—Sophia-Antipolis, Valbonne, France. He is currently a member of the Brain-Inspired Computing Group, IBM Research—Almaden, San Jose, CA, USA, where he is involved in large-scale simulations using the IBM Neuro Synaptic Core Simulator (Compass) as part of the DARPA-funded SyNAPSE Project. He is involved in the development of the Corelet programming language, for programming the reconfigurable neuromorphic hardware, and algorithms and applications with networks of neurosynaptic cores for building cognitive systems. He has authored works in several international journals and conferences. His current research interests include neuromorphic architecture and simulations, high performance and distributed computing, machine learning, optimization techniques, and graph theory.

Dr. Datta is a member of ACM.

Gi-Joon Nam (S'99–M'01–SM'06) received the B.S. degree in computer engineering from Seoul National University, Seoul, Korea, and the M.S. and Ph.D degrees in computer science and engineering from the University of Michigan, Ann Arbor, MI, USA, in 2001.

From 2001 to 2013, he was with the IBM Research—Austin, Austin, TX, USA, in the physical design space, particularly placement and timing closure flow. Since 2014, he has been managing physical synthesis department primarily targeted for IBM's P and Z microprocessors and system application-specific integrated circuit designs. His current research interests include computer-aided design algorithms, combinatorial optimization, very large-scale integration system designs, and computer architecture.

Dr. Nam is a Senior Member of ACM.

Brian Taba received the B.S. degree in electrical engineering from Caltech, Pasadena, CA, USA, in 1999, and the Ph.D. degree in bioengineering (neuromorphic engineering) from the University of Pennsylvania, Philadelphia, PA, USA, under the supervision of Dr. K. Boahen, in 2005.

Since 2013, he has been a Software Engineer with the Brain-Inspired Computing Group, IBM. His current research interests include computation and neural systems, a multidisciplinary approach to study the brain that combines engineering with neurobiology.

Michael Beakes (M'94) is a Senior Scientist with IBM's T. J. Watson Research Center, Yorktown Heights, NY, USA. Throughout his career, he has concurrently played the roles of a designer, methodology/tool developer, and team leader for the custom analog/mixed-signal integrated circuit design community. He is also a mentor and an advisor for grade school and high school technology and robotics programs. Michael received bachelor degrees in physics and electrical engineering, and has a masters degree in computer engineering.

Bernard Brezzo receive the Diploma degree in electronics engineering from Conservatoire National des Arts et Metiers, Paris, France.

He is a Senior Engineer with IBM's T.J. Watson Research Center, Yorktown Heights, NY, USA. His current research interests include high-performance computing, FPGA emulation, and IBM cognitive computing as part of the DARPA's SyNAPSE project.

Jente B. Kuang (SM'01) received the B.S. degree in electrical engineering from National Taiwan University, Taipei, Taiwan, in 1982, the M.S. degree in electrical engineering from Columbia University, New York, NY, USA, in 1986, and the Ph.D. degree in electrical engineering from Cornell University, Ithaca, NY, USA, in 1990, respectively.

From 1982 to 1984, he served as an Electronics Instructor with the Electronics Training Center, Army Missile Command, Taiwan. He was a Research Assistant with the Compound Semiconductor Research Group and National Submicron Facility, Cornell University from 1986 to 1990. He joined IBM in 1990. Since then, he has been researching on a variety of research and product development projects for bipolar, BiCMOS, and CMOS devices and circuits, including the design of flash memories, SRAMs, register files, and high-speed low-power digital circuits. He was a Research Staff Member with IBM Austin Research Laboratory, Austin, TX, USA, until 2015. In 2015, he joined PowerCore Technology Corporation, the U.S. Subsidiary of Suzhou PowerCore Company Ltd., as the President and the Director of the design center located in Austin, TX, USA.

Rajit Manohar (SM'97) received the B.S., M.S., and Ph.D. degrees from Caltech, Pasadena, CA, USA, in 1994, 1995, and 1998, respectively.

He has been with Cornell University, Ithaca, NY, USA, since 1998, where he is currently a Professor of Electrical and Computer Engineering, and where his group conducts research on asynchronous design. He founded Achronix Semiconductor to commercialize high-performance asynchronous FPGAs.

Dr. Manohar was a recipient of an NSF CAREER award, six best paper awards, seven teaching awards, and was named one of MIT Technology Review's top 35 young innovators under 35 for his contributions to low power microprocessor design.

William P. Risk (M'86) currently serves as a Technical Project Manager for the DARPA SyNAPSE project. In addition to helping manage the program, he works on visualization neurosynaptic simulations and neuromorphic chip behavior. For this purpose, he built BrainWall, a 65-million pixel display system. Prior to joining the SyNAPSE program, he was a Research Staff Member at IBM Research—Almaden, San Jose, CA, USA, specializing in laser and optics.

Bryan Jackson received the Advanced B.A. degree from Occidental College, Los Angeles, CA, USA, and the Ph.D. degree from the University of California, Berkeley, Berkeley, CA, USA.

He joined IBM in 2008 under the DARPA funded SyNAPSE program. In 2010, he became a Research Staff Member and Technical Project Manager for Hardware Design in the Cognitive Computing (SyNAPSE) Group at IBM Research—Almaden, San Jose, CA, USA. In 2013, he became Manager of the Cognitive Computing Hardware Team. This team is tasked with designing, building, and testing new silicon architectures under the SyNAPSE program.

He is currently manager of IBM's Brain Inspired Hardware Group at IBM Research—Almaden. In this role, he led the team of hardware engineers that designed, built, and tested the world's largest neuromorphic processor, TrueNorth.

Dharmendra S. Modha (M'92–F'11) received the B.Tech. degree in computer science and engineering from IIT Bombay, Maharashtra, India, and the Ph.D. degree in electrical and computer engineering from the University of California at San Diego, San Diego, CA, USA.

He is an IBM Chief Scientist for brain-inspired computing. He has made significant contributions to IBM businesses via innovations in caching algorithms for storage controllers, clustering algorithms for services, and coding theory for disk drives. He is an IBM Master Inventor. He is a cognitive computing pioneer and leads a highly successful effort to develop brain-inspired computers. His work has been featured in *The Economist*, *Science*, *New York Times*, *Wall Street Journal*, *The Washington Post*, *BBC*, *CNN*, *PBS*, *Discover*, *MIT Technology Review, Associated Press, Communications of the ACM*, IEEE SPECTRUM, *Forbes, Fortune,* and *Time*, amongst others. His work has been featured on covers of *Science* (twice), *Communications of the ACM*, and *Scientific American*. He has authored over 60 papers and holds over 100 patents.

Dr. Modha was a recipient of the ACM's Gordon Bell Prize, USENIX/FAST Test of Time Award, best paper awards at ASYNC and IDEMI, First Place in the Science/NSF International Science and Engineering Visualization Contest, IIT Bombay Distinguished Alumni Award, and was a Runner-Up for the 2014 Science Breakthrough of the Year. In 2013 and 2014, he was named the Best of IBM. EE Times named him amongst 10 Electronic Visionaries to watch. At IBM, he received the Pat Goldberg Memorial Best Paper award twice, an Outstanding Innovation Award, an Outstanding Technical Achievement Award, and a Communication Systems Best Paper Award. In 2010, he was elected to the IBM Academy of Technology. In 2014, he was appointed an IBM Fellow. He is a fellow of the World Technology Network.